

软件使用帮助手册

第一章 MARS

MARS (the Mips Assembly and Runtime Simulator), Mips 汇编程序运行模拟器, 可以通过命令行和集成开发环境两种途径运行。该软件主要用于模拟真实 Mips 处理器, 运行 Mips 汇编程序。用户通过查看运行结果检验汇编程序的正确性。软件运行需要 java 运行环境。

在 MARS 的集成开发环境中, 用户可以编辑或导入 Mips 汇编程序, 汇编, 并运行或调试程序。用户可以轻易设置、移除断点, 并向前、向后分步执行, 或者在运行过程中修改寄存器和内存内容。

1 下载安装

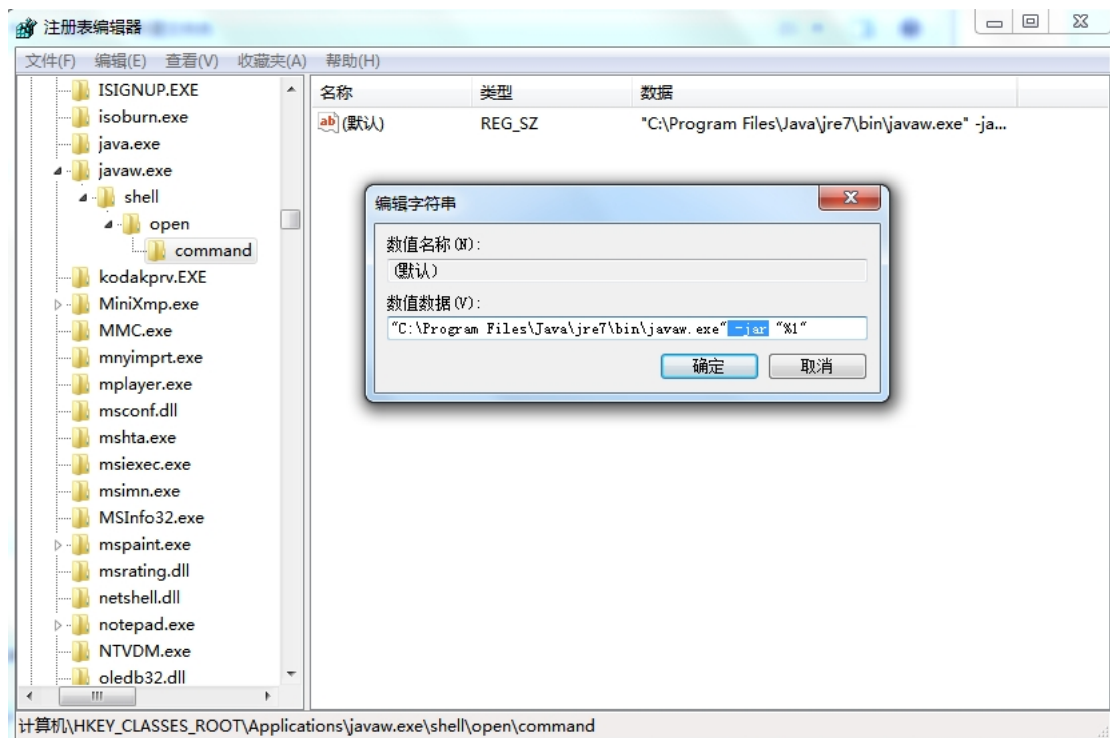
下载 MARS。到 <http://courses.missouristate.edu/KenVollmar/MARS/download.htm> 下载最新版。

配置 java 运行环境。到网上搜索“java se download”, 下载 JRE7, 把 java 运行环境配置好。下载地址:

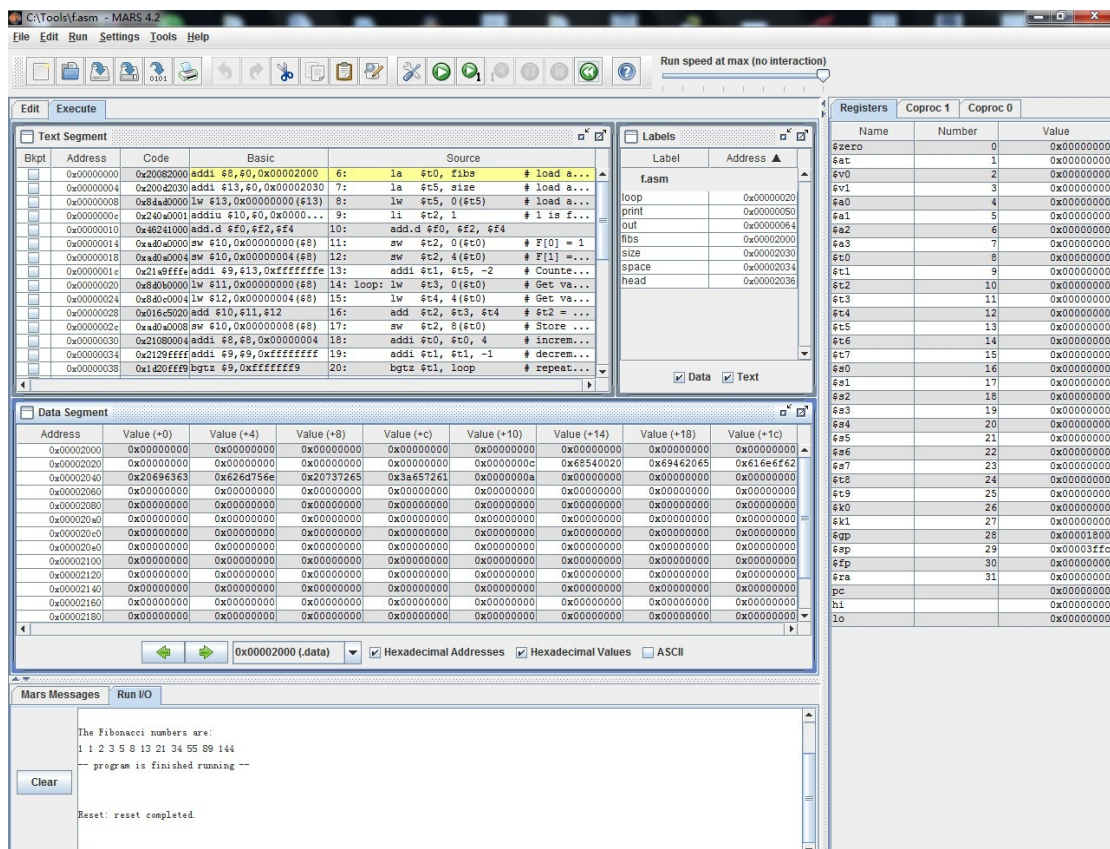
<http://www.oracle.com/technetwork/java/javase/downloads/jre7-downloads-1880261.html>

配置双击打开 MARS 软件包。MARS 是一个可直接运行的 JAR 文件, 无需安装。可参照 <http://jingyan.baidu.com/article/200957617c3619cb0621b44d.html> 配置, 通过双击.jar 文件打开软件。

- a. 双击需要运行的 jar 文件或在 jar 文件上点击鼠标右键, 在弹出的菜单中选择“打开方式”→“选择默认程序”。
- b. 在“打开方式”设置窗口中点击选中选项“始终使用选择的程序打开这种文件”前的复选框, 再点击旁边的“浏览”按钮。
- c. 选择 JAVA 虚拟机的安装文件夹, 如果安装的是 Java 7, 安装文件夹一般为“C:\Program Files\Java\jre7\bin”(具体文件夹请自行查找), 找到 javaw.exe 文件, 点击“打开”按钮。
- d. 这时已经设置了使用 javaw.exe 应用程序来打开 jar 文件, 可是实际上这时双击 jar 文件还是没有反应, 这是因为没有设置打开文件参数导致的。
- e. 在 Windows 开始菜单的搜索框中输入“regedit”, 在上方搜索出的文件 regedit 上点击鼠标右键, 在弹出的菜单中选择“以管理员身份运行”。
- f. 在注册表编辑器中, 找到“HKEY_CLASSES_ROOT\Applications\javaw.exe\shell\open\command”, 在其中文件打开命令中加入参数“-jar”(无引号), 修改后的数值类似: “C:\Program Files\Java\jre7\bin\javaw.exe -jar "%1””(只需要添加-jar 参数, 无需修改其他信息), 保存并退出注册表编辑器。



2 用户界面



菜单和工具栏：在这找到最常用的工具，包括新建、导入文件，汇编，执行，单步等。不知道按钮代表什么操作，把鼠标箭头移到按钮上会有提示

编辑窗口：编辑 Mips 汇编代码

执行窗口：汇编之后进入运行状态，执行窗口会有信息。该窗口包括三部分：代码段（显示要运行的代码，包括汇编和二进制形式以及地址）、数据段（显示用到的数据在内存中的值）和标签栏（显示跳转的标签地址）

代码段：显示要运行的代码，包括汇编和二进制形式以及地址

数据段：显示用到的数据在内存中的值

标签栏：显示跳转的标签地址

消息区域：执行反馈，包括程序结果和错误信息等

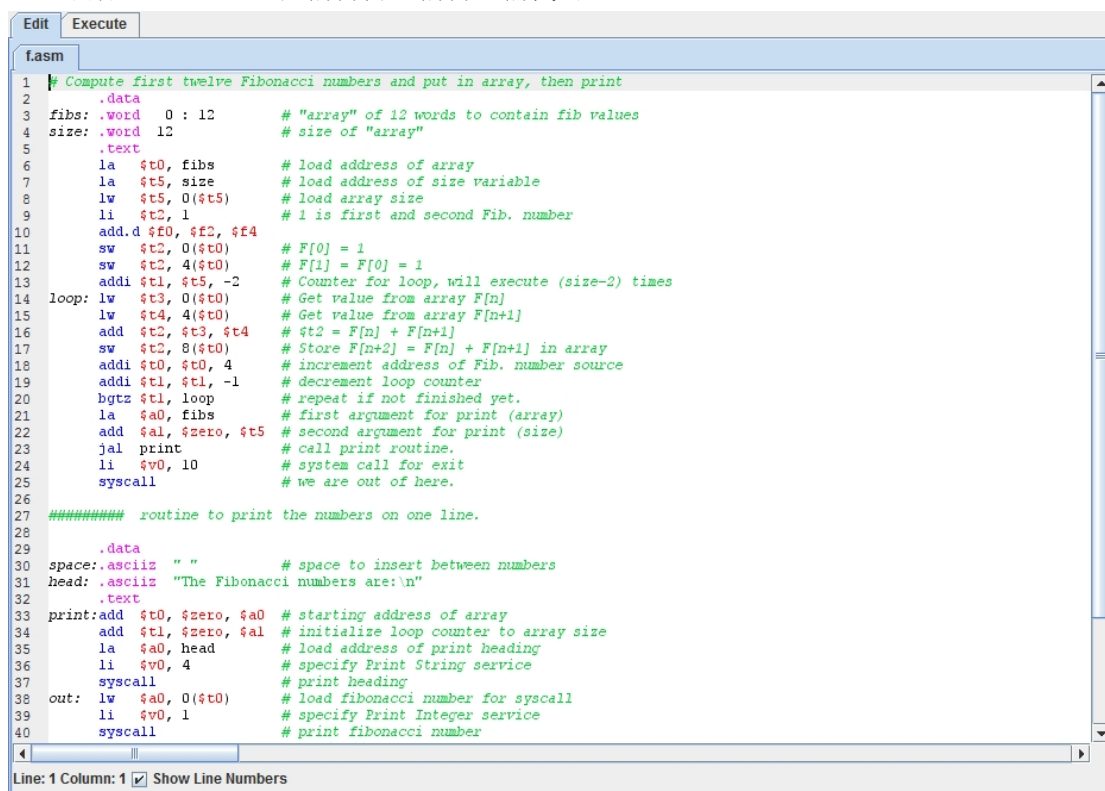
MIPS 寄存器：显示 Mips 处理器 32 个通用寄存器以及 PC 的值

3 基本使用

3.1 导入汇编程序

File->Open，找到汇编程序，打开。

或者 File->New，在编辑窗口编辑汇编代码。



```
1  # Compute first twelve Fibonacci numbers and put in array, then print
2  .data
3  fibs: .word 0 : 12      # "array" of 12 words to contain fib values
4  size: .word 12         # size of "array"
5  .text
6  la $t0, fibs           # load address of array
7  la $t5, size            # load address of size variable
8  lw $t5, 0($t5)          # load array size
9  li $t2, 1              # 1 is first and second Fib. number
10 add.d $f0, $f2, $f4
11 sw $t2, 0($t0)          # F[0] = 1
12 sw $t2, 4($t0)          # F[1] = F[0] = 1
13 addi $t1, $t5, -2       # Counter for loop, will execute (size-2) times
14 loop: lw $t3, 0($t0)     # Get value from array F[n]
15       lw $t4, 4($t0)     # Get value from array F[n+1]
16       add $t2, $t3, $t4   # $t2 = F[n] + F[n+1]
17       sw $t2, 8($t0)     # Store F[n+2] = F[n] + F[n+1] in array
18       addi $t0, $t0, 4    # increment address of Fib. number source
19       addi $t1, $t1, -1   # decrement loop counter
20       bgtz $t1, loop      # repeat if not finished yet.
21       la $a0, fibs       # first argument for print (array)
22       add $a1, $zero, $t5 # second argument for print (size)
23       jal print          # call print routine.
24       li $v0, 10         # system call for exit
25       syscall            # we are out of here.
26
27 ##### routine to print the numbers on one line.
28
29 .data
30 space: .asciiz " "       # space to insert between numbers
31 head: .asciiz "The Fibonacci numbers are:\n"
32 .text
33 print: add $t0, $zero, $a0 # starting address of array
34       add $t1, $zero, $a1 # initialize loop counter to array size
35       la $a0, head        # load address of print heading
36       li $v0, 4           # specify Print String service
37       syscall            # print heading
38 out:  lw $a0, 0($t0)       # load fibonacci number for syscall
39       li $v0, 1           # specify Print Integer service
40       syscall            # print fibonacci number
```

3.2 汇编

Run->Assemble，或者单击 

如果代码有错，在信息窗口会有提示，根据提示修改代码。否则正常进入执行窗口。

3.3 完整运行

Run->Go，或者单击 ，或者 F5

3.4 分步运行

Run->Step, 或者单击 , 或者 F7

3.5 设置断点

在代码段最左侧的复选框中打钩设置断点。当程序运行到断点会自动停止。取消打钩会取消断点。

3.6 检验结果

可通过三种途径检查程序的正确性。一是看寄存器状态, 查看寄存器中的值是否跟预想的一致; 二是看内存值, 看程序是否影响了特定的内存地址; 三是程序的文本输出, 在消息窗口中看。

第二章 Logisim

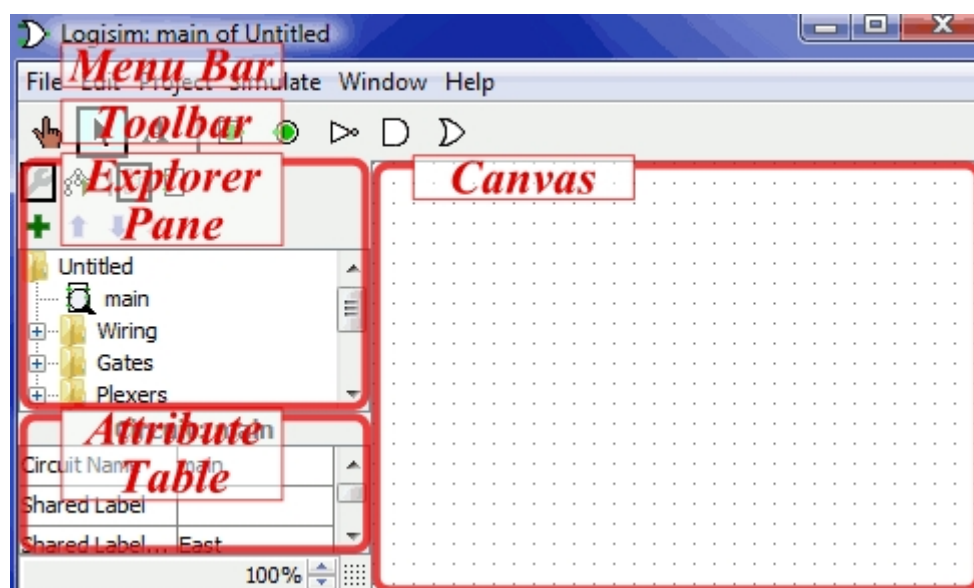
Logisim 是一款用于帮助学生设计和模拟数字逻辑电路的辅助教学软件。运用 logisim 提供的工具，不仅可以设计相应的数字逻辑电路，还可模拟电路运行，验证电路设计的正确性。使用 Logisim，大型复杂的数字逻辑电路设计不再复杂。采用从下向上分层设计的思路，学生先设计实现小部件，验证通过后，再将小部件放到大设计中去。如此，设计和模拟完整的 CPU 也不是问题。

1 下载安装

下载  logisim-win-2.7.0.zip

无需安装，双击  logisim-win-2.7.0.exe 即可运行

2 用户界面



2.1 浏览栏（Explorer Pane）

在浏览栏，学生可以浏览到当前的项目设计和加载的工具库。设计好的或正在设计的数字电路在项目名下边，main 代表项目的顶层设计；以文件夹形式组织的是设计电路过程中需要用到的工具，包括连线、逻辑门、编解码器、算术运算器、存储单元、输入输出和基本的操作工具。

- **Wiring:** Components that interact directly with wires.
- **Gates:** Components that perform simple logic functions.
- **Plexers:** More complex combinational components, like multiplexers and decoders.
- **Arithmetic:** Components that perform arithmetic.
- **Memory:** Components that remember data, like flip-flops, registers, and RAM.
- **I/O:** Components that exist for the purpose of interacting with the user.
- **Base:** Tools that are integral to using Logisim, though you probably won't need to dig into this library very often.

2.2 属性栏 (Attribute Table)

使用  选择具体器件，可在属性栏修改该器件的属性，包括数据宽度、器件大小、朝向、器件名等。

2.3 工具栏 (Toolbar)

工具栏只放置了一些最基本的工具（箭头、文字输入、输入输出、与或非门），这些在工具库中都能够找到。

3 基本使用

3.1 根据提供的与或非门设计异或门器件

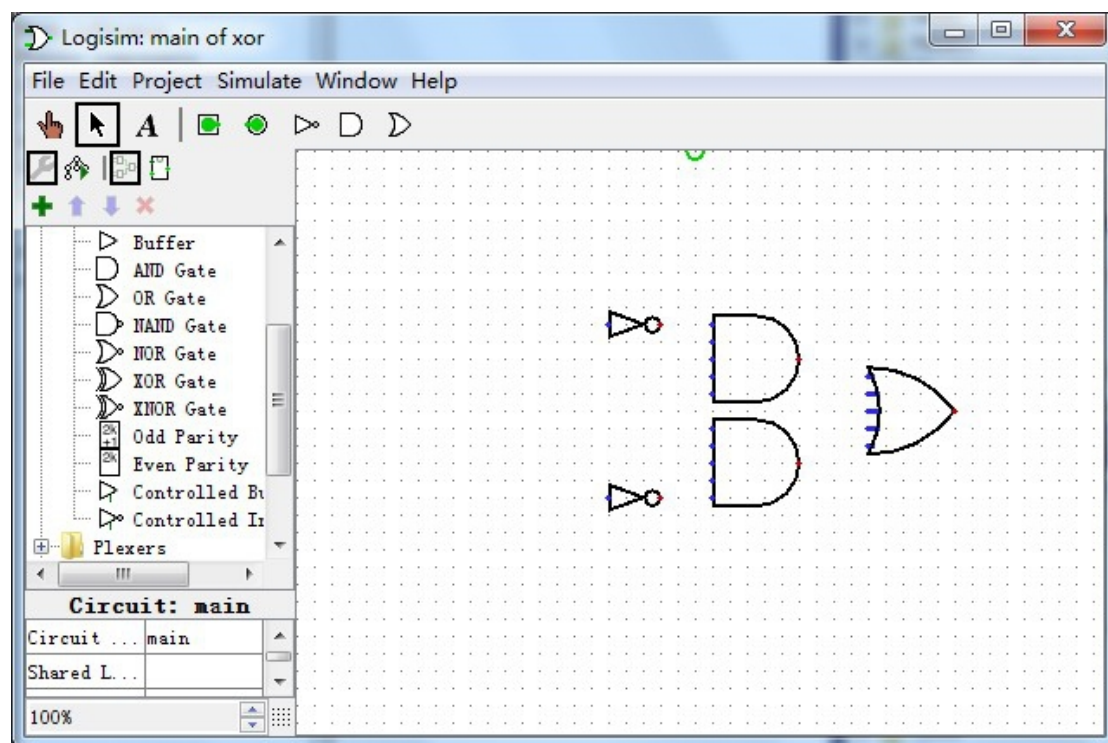
3.1.1 新建项目

File->New, 新建项目

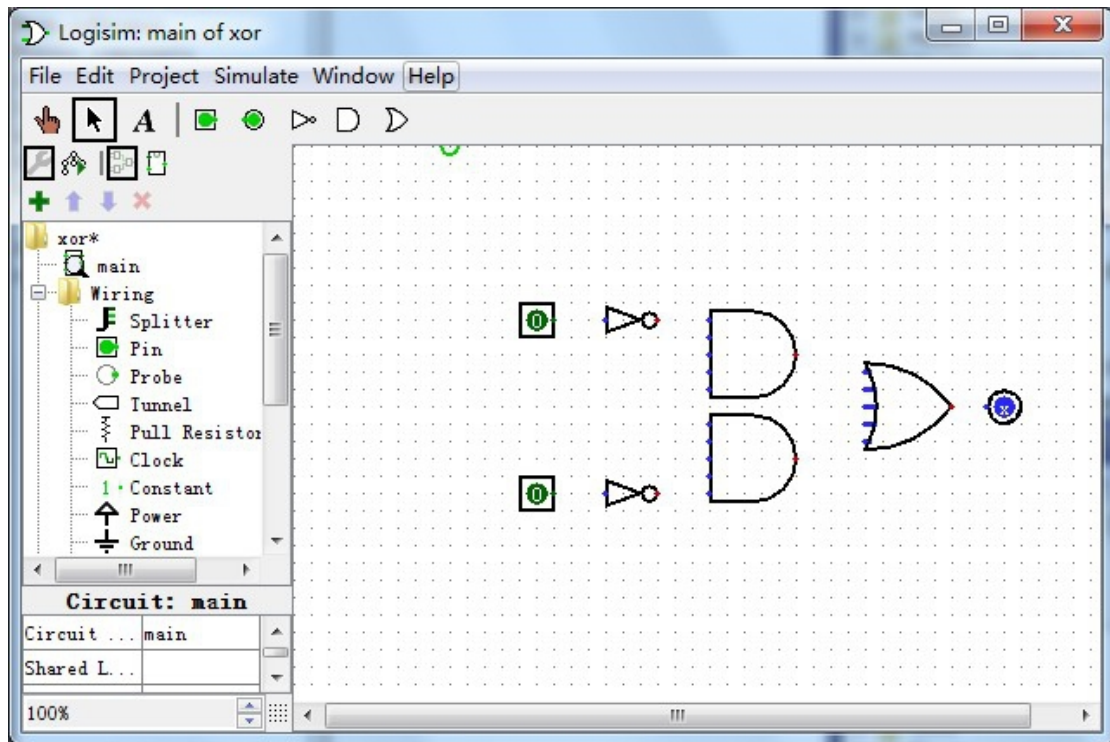
File->Save, 将新项目命名为 xor.circ, 并保存

3.1.2 设计器件

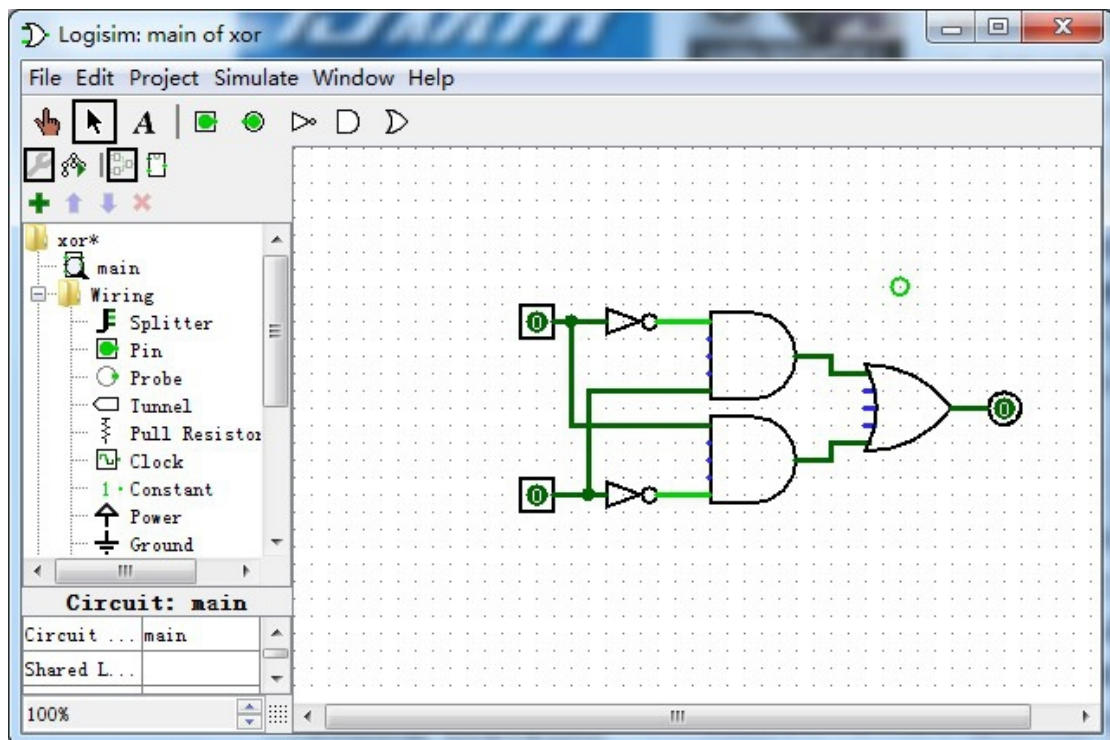
添加门器件。在浏览栏中选择与门、或门和非门，按照需要将它们添加到右边的画布。可直接从工具栏选择。



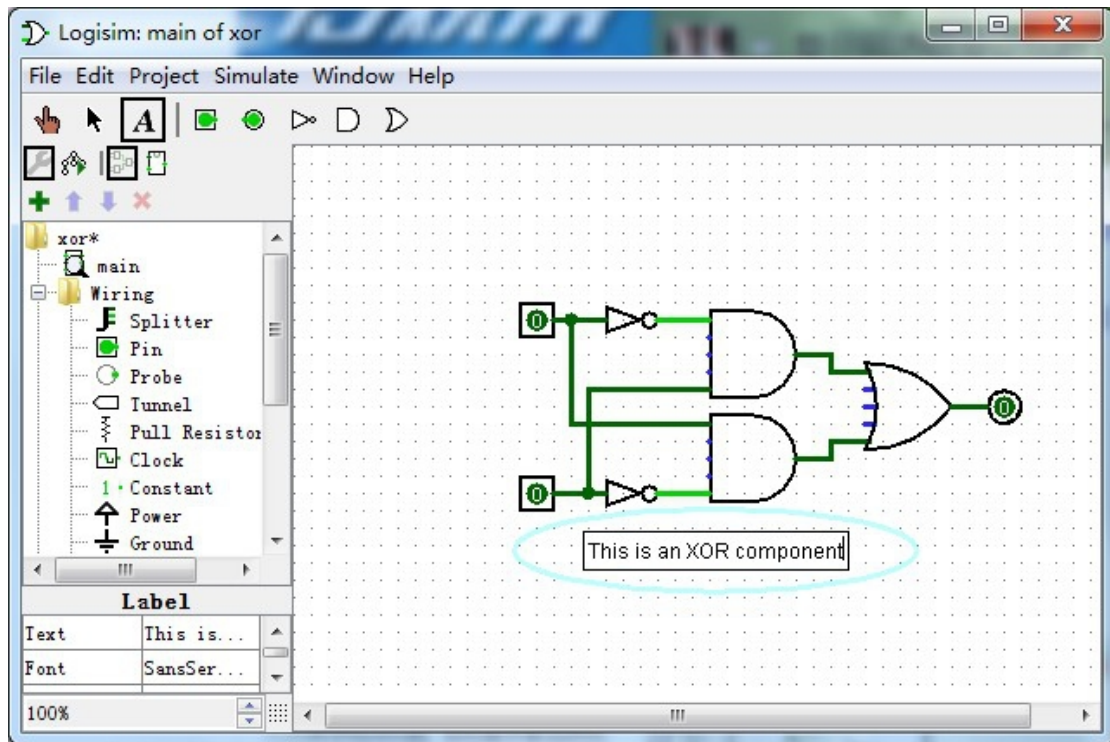
添加输入输出引脚。在连线器件中选择 pin, 将其添加到画布。可直接从工具栏选择。



连线。箭头选择 ，将箭头放在器件连线端，单击，按住左键拖动即可将器件连接起来。



添加文字。选择工具 ，在画布中器件下边单击输入 “This is an XOR component”。



3.1.3 电路验证

箭头选择 ，单击可以改变输入引脚的值，查看输出引脚的值，判断跟预想的是否一致。在上图中， $0 \text{ xor } 0 = 0$ ，没问题。再试一下其它的值，查看输出，验证电路的正确性。在电路中，亮色的连线代表线值为 1，暗色的线值为 0。

3.2 设计 4X32 寄存器文件

4 个 32 位寄存器。可复位；一个写端口，写使能有效时，时钟上升沿写有效；两个读端口，写使能无效时读出相应寄存器数据。

3.2.1 单个寄存器设计

新建项目，命名为 regfile.circ

在浏览栏，右键 regfile，添加电路 register

双击 register，在其画布中设计单个寄存器

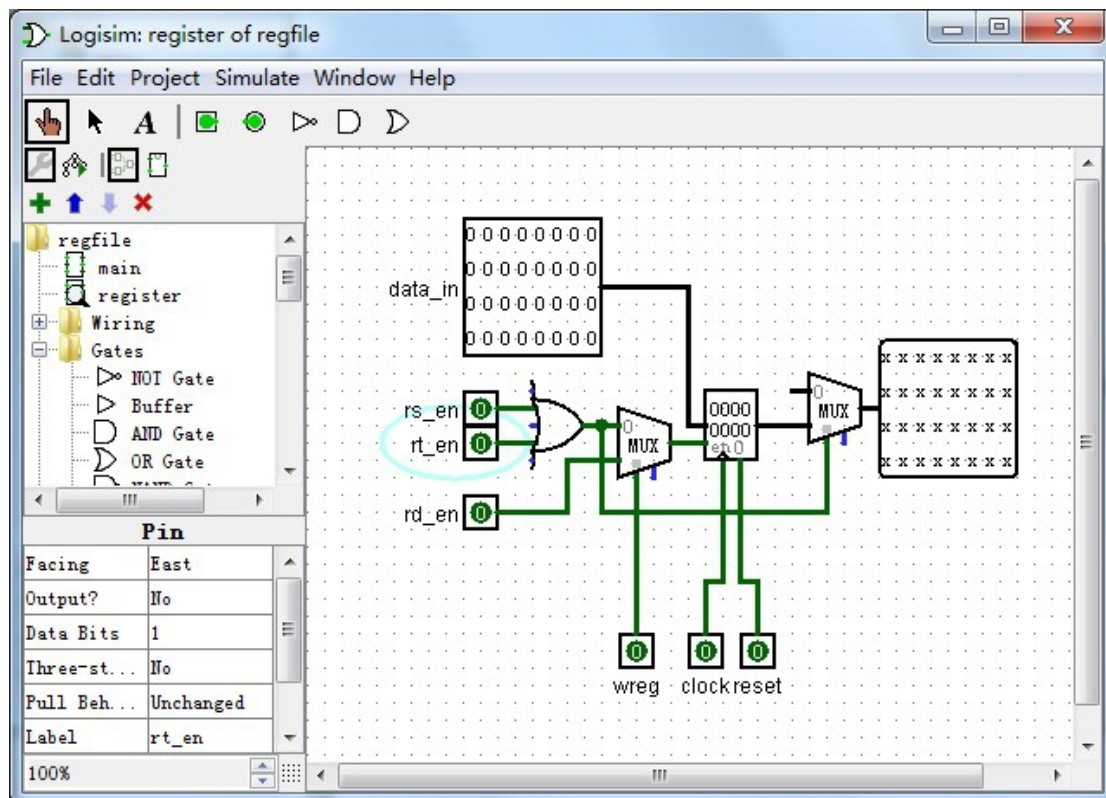
在 Memory 中选择 Register，添加到画布，设置数据宽度为 32 位

添加 32 位数据输入端，命名为 data_in

添加 clock 和 reset 输入端

设计 enable 输入。根据 enable 为 0 时，读有效，enable 为 1 时，写有效，需要在 enable 前端放置一个选择器，选择信号为写使能信号 wreg，输入为读有效和写有效信号。因有两个读寄存器端口，会产生两个读有效信号，只要其中任一个有效，都会读出数据，因此需要添加一个或门。

对于输出端。只有当读有效时才输出数据，否则输出任意值。因此需要添加一个选择器，选择信号为读有效信号。32 位输出端肯定也是需要的。



3.2.2 单个寄存器验证

数据写入。将数据设置为 0x01010101, rd_en, wreg 为 1, 给个时钟上升沿, 看寄存器值是否变成 0x01010101

数据读取。将 rs_en 或 rt_en 设为 1, wreg 为 0, 查看输出端是否跟寄存器内容一致

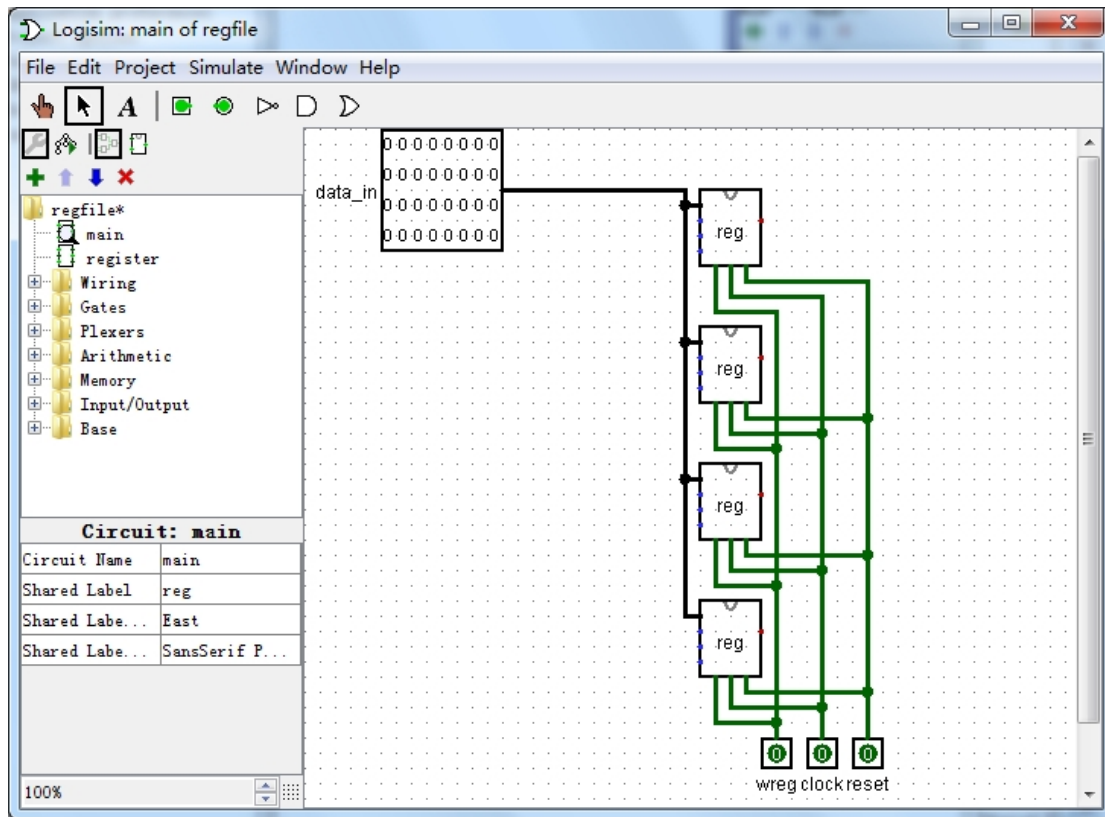
复位。单击 reset, 看寄存器内容是否置 0

3.2.3 寄存器文件设计

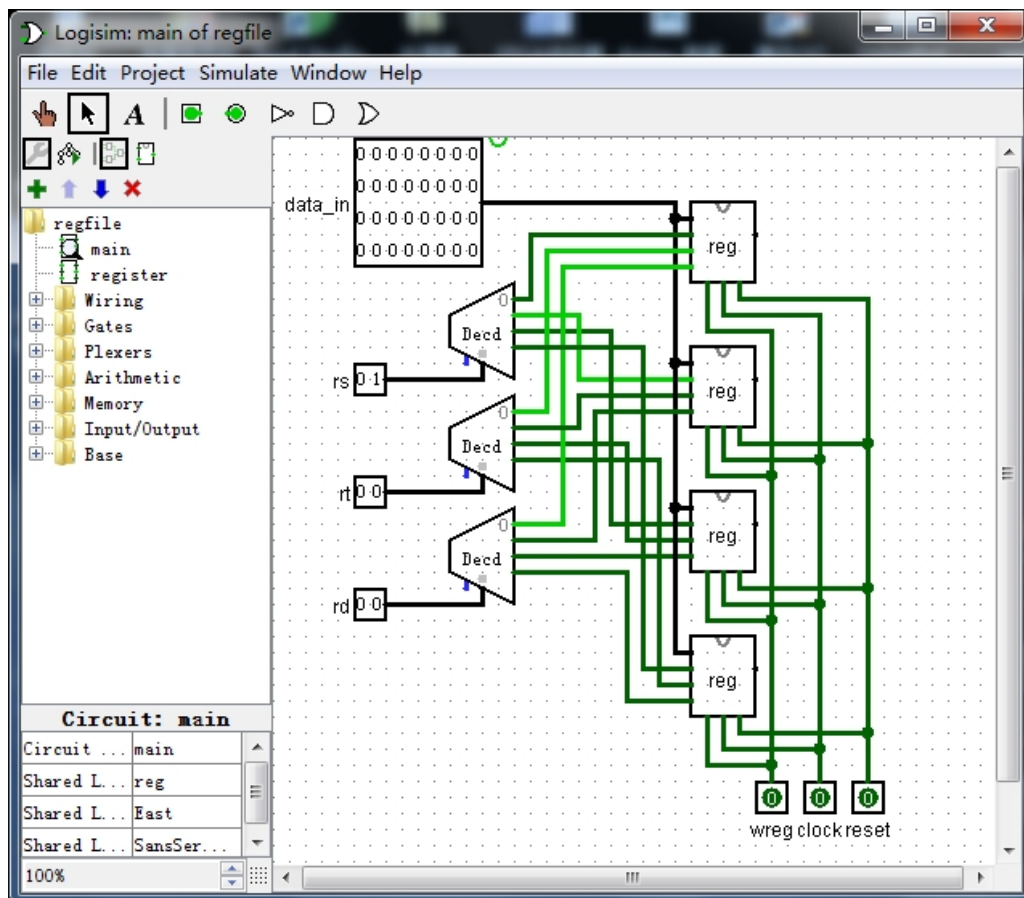
单个寄存器验证通过后, 可以将其作为一个封装好的元器件使用。

在 main 中添加 4 个自定义的 register。

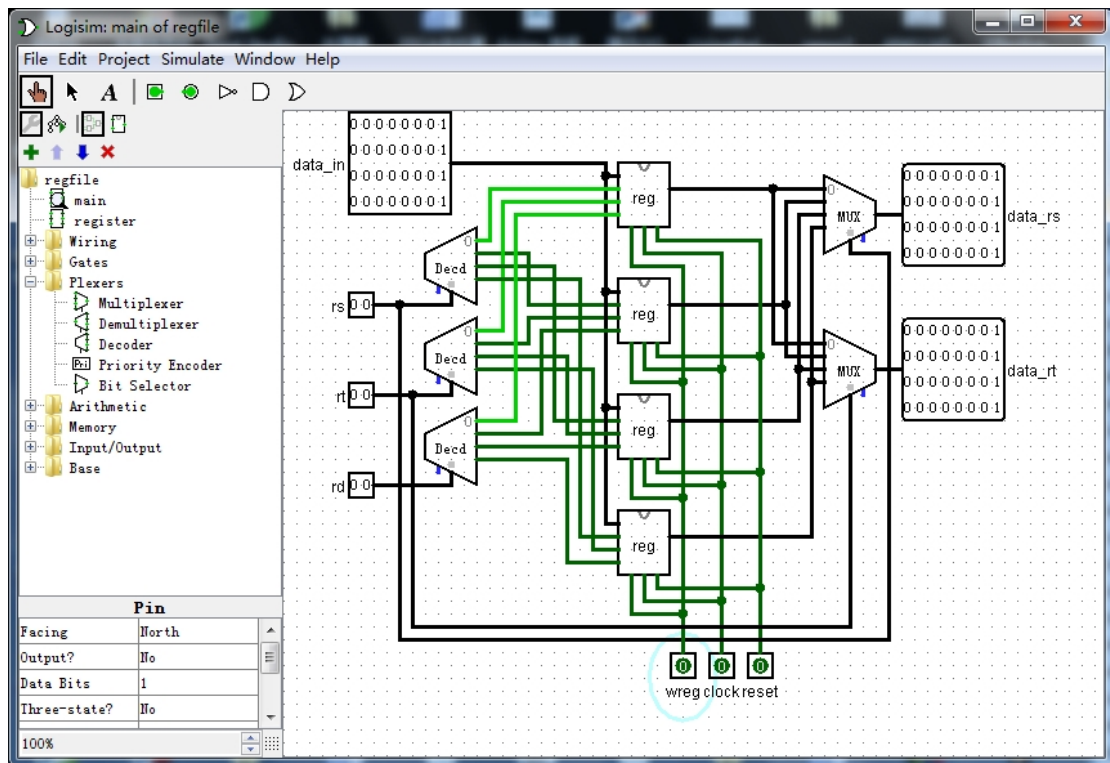
添加数据输入引脚、写使能、时钟和复位引脚。



根据输入的寄存器端口号译码得到相应寄存器的 enable 输入。



输出端口的数据来自哪个寄存器，需要选择器选择，选择信号为相应的寄存器端口号。



3.2.4 寄存器文件验证

同单个寄存器验证思路验证该寄存器文件电路的正确性。

第三章 Xilinx ISE

ISE 的主要功能包括设计输入、综合、仿真、实现和下载，涵盖了 FPGA 开发的全过程，从功能上讲，其工作流程无需借助任何第三方 EDA 软件。

- 设计输入: ISE 提供的设计输入工具包括用于 HDL 代码输入和查看报告的 ISE 文本编辑器 (The ISE Text Editor)，用于原理图编辑的工具 ECS (The Engineering Capture System)，用于生成 IP Core 的 Core Generator，用于状态机设计的 StateCAD 以及用于约束文件编辑的 Constraint Editor 等。

- 综合: ISE 的综合工具不但包含了 Xilinx 自身提供的综合工具 XST，同时还可以内嵌 Mentor Graphics 公司的 LeonardoSpectrum 和 Synplicity 公司的 Synplify，实现无缝链接。

- 仿真: ISE 本身自带了一个具有图形化波形编辑功能的仿真工具 HDL Benchner，同时又提供了使用 Model Tech 公司的 Modelsim 进行仿真的接口。

- 实现: 此功能包括了翻译、映射、布局布线等，还具备时序分析、管脚指定以及增量设计等高级功能。

- 下载: 下载功能包括了 BitGen，用于将布局布线后的设计文件转换为位流文件，还包括了 ImPACT，功能是进行设备配置和通信，控制将程序烧写到 FPGA 芯片中去。

- 使用 ISE 进行 FPGA 设计的各个过程可能涉及到的设计工具如下表所示。

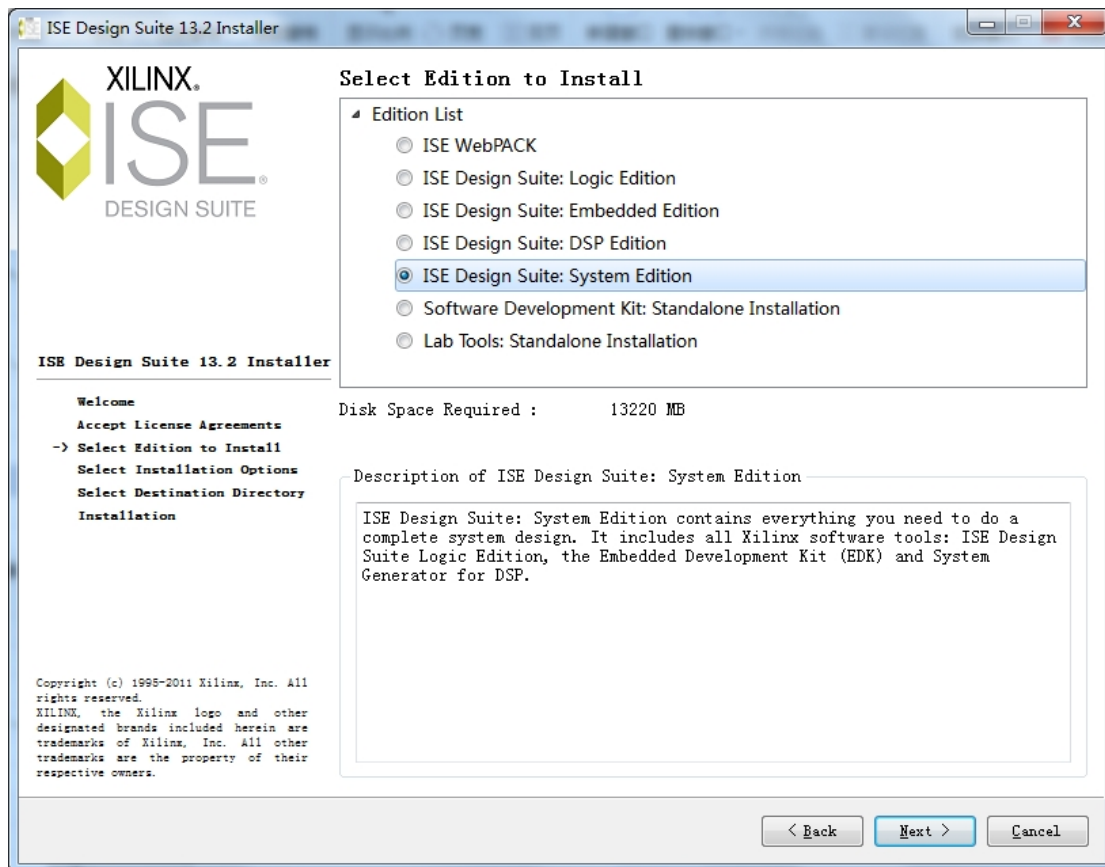
设计输入	综合	仿真	实现	下载
HDL文本编辑器	XST		Translate	
ECS原理图编辑器	FPGA Express	HDL Benchner	MAP	BitGen
StateCAD状态机编辑器	(Synplify	(Model Sim)	Place and Route	IMPACT
Core Generator	LeonardoSpectrum)		Xpower	
Constraint Editor				

1 下载安装

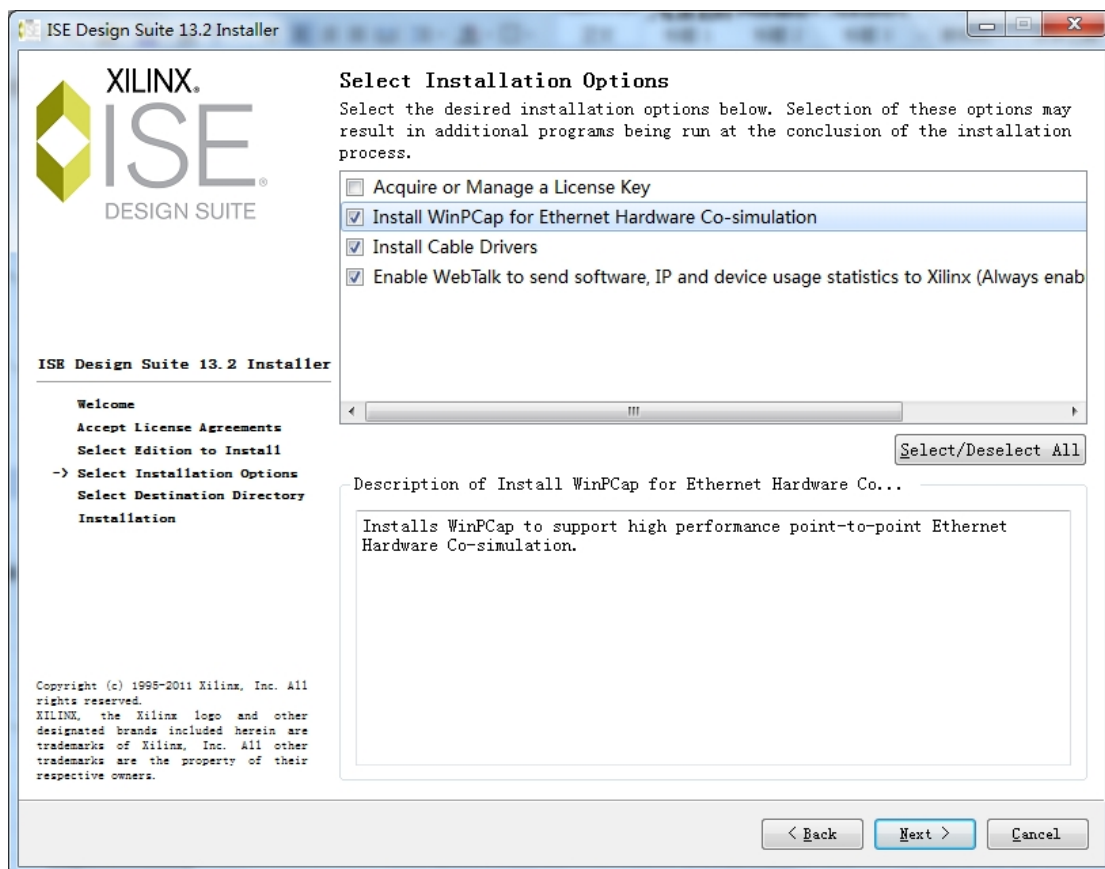
1.1 下载安装 ISE

进入 Xilinx 官网点击顶部的“Sign in”注册账号，并登陆。登陆后选择顶部的“downloads”，左下角选择“ISE Design Tools”可以选择 ISE 13.2 版本进行下载。

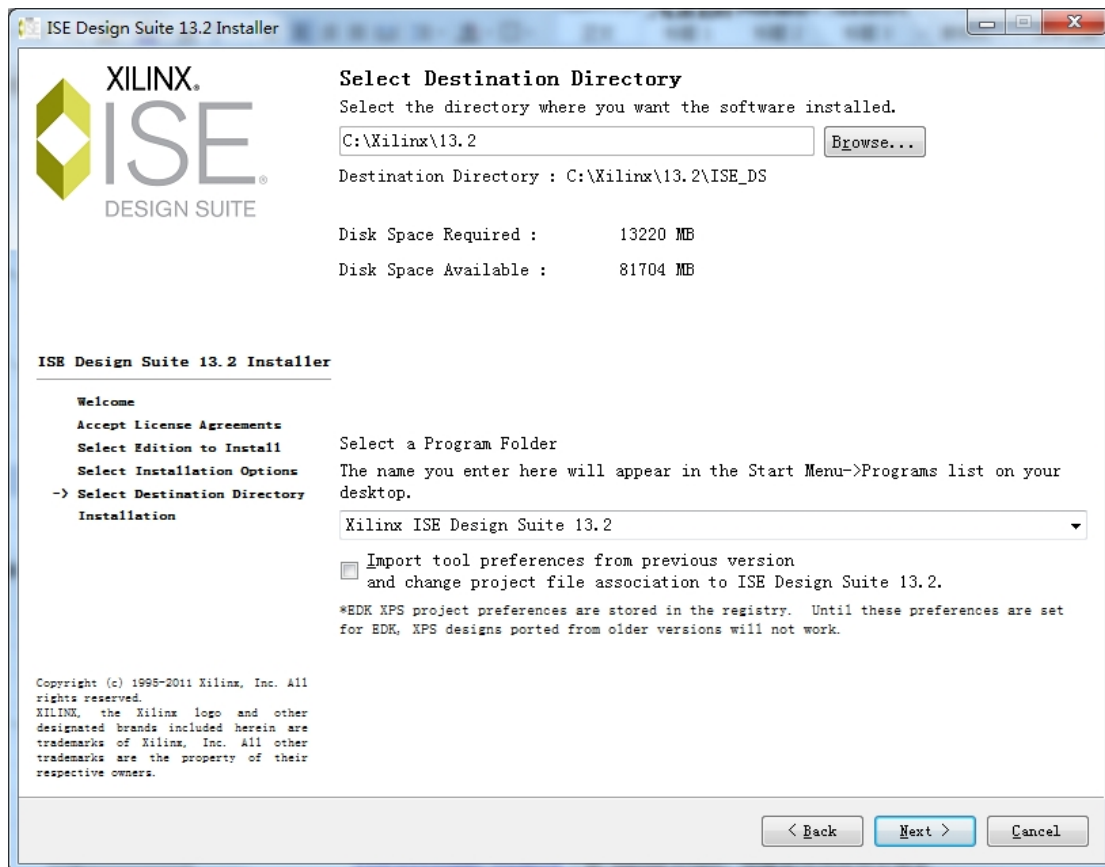
安装 ISE，打开安装文件，选择"System Edition"，选择路径安装（不要安装在中文或者有空格的目录下）。



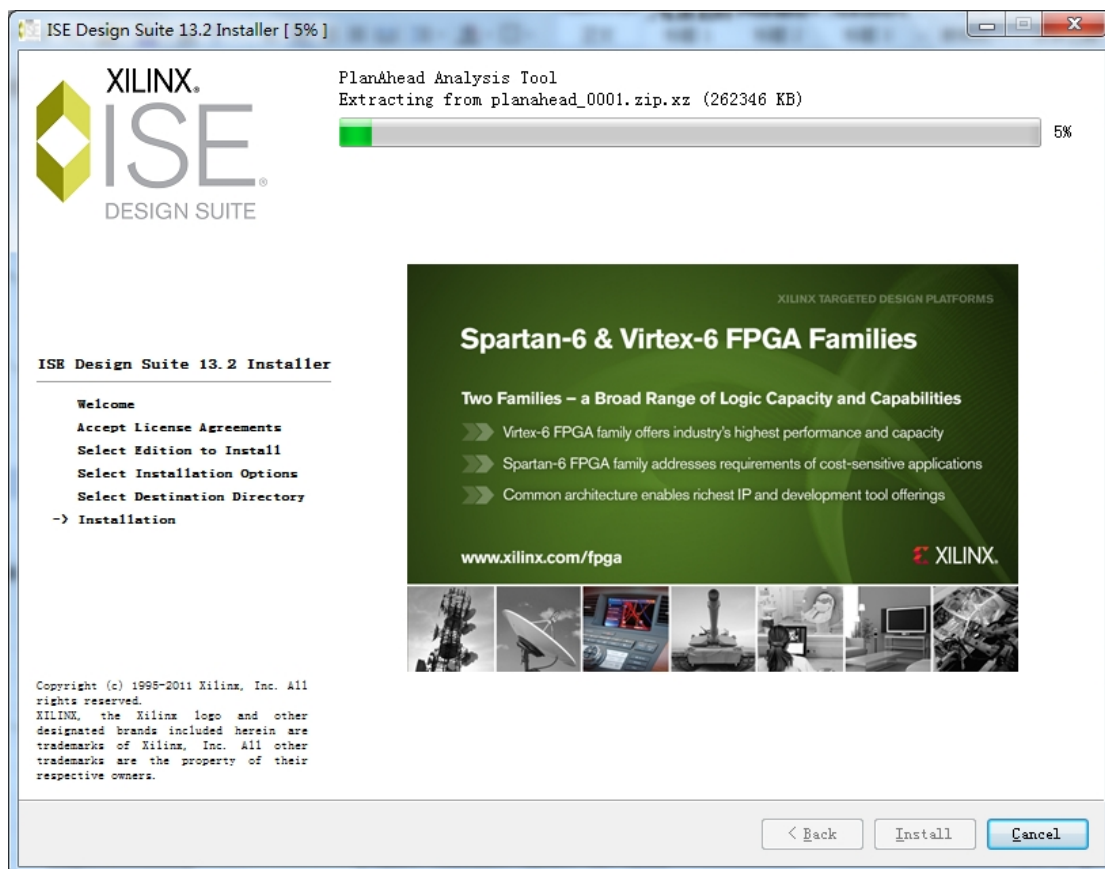
将获取 license 的勾取消，不去获取 license。



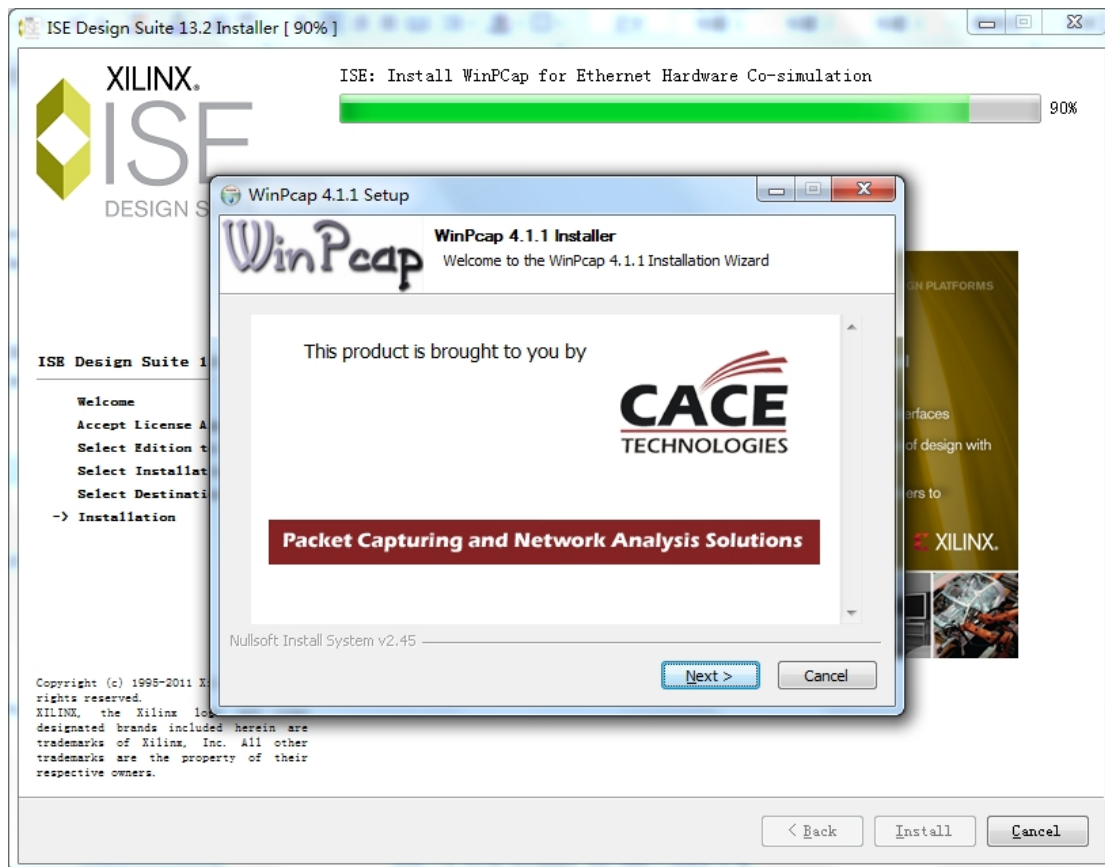
默认安装路径，将复选框中的勾取消。



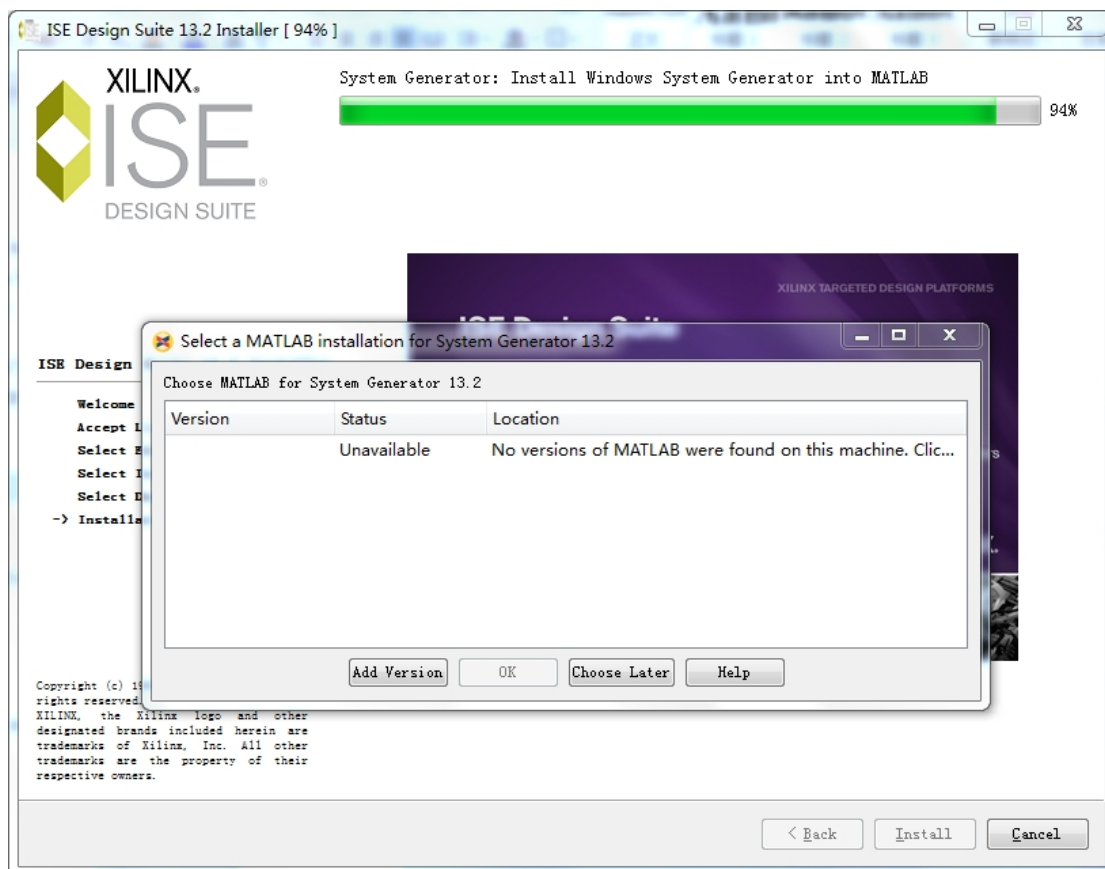
正在安装。



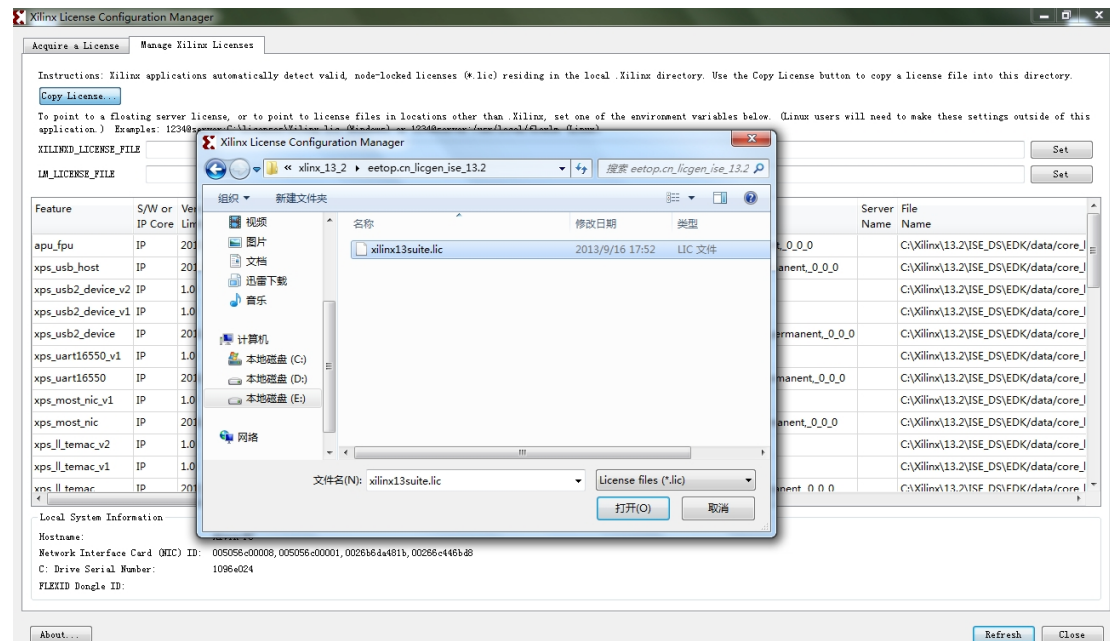
安装 Wincap。



不需要 MATLAB，选择“Choose Later”。



导入 license。找到  eetop.cn_license_is13.2.rar，解压后单击 exe 文件可以生成.lic 文件，在 ISE 软件中（Help->Manage license...）导入 license 文件即可正常使用 ISE。



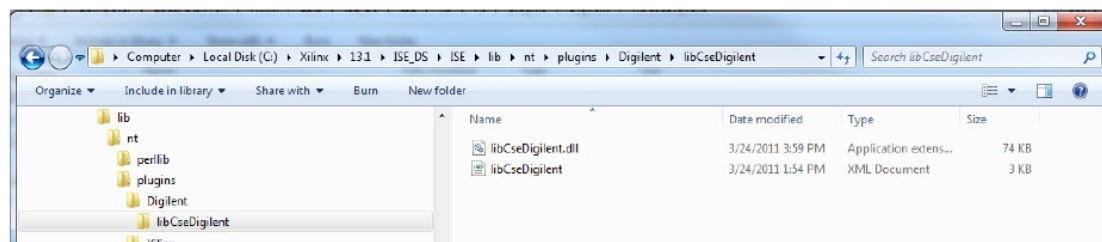
1.2 下载器支持

为了让 ISE 软件识别 USB 下载线，需要进行必要的设定。

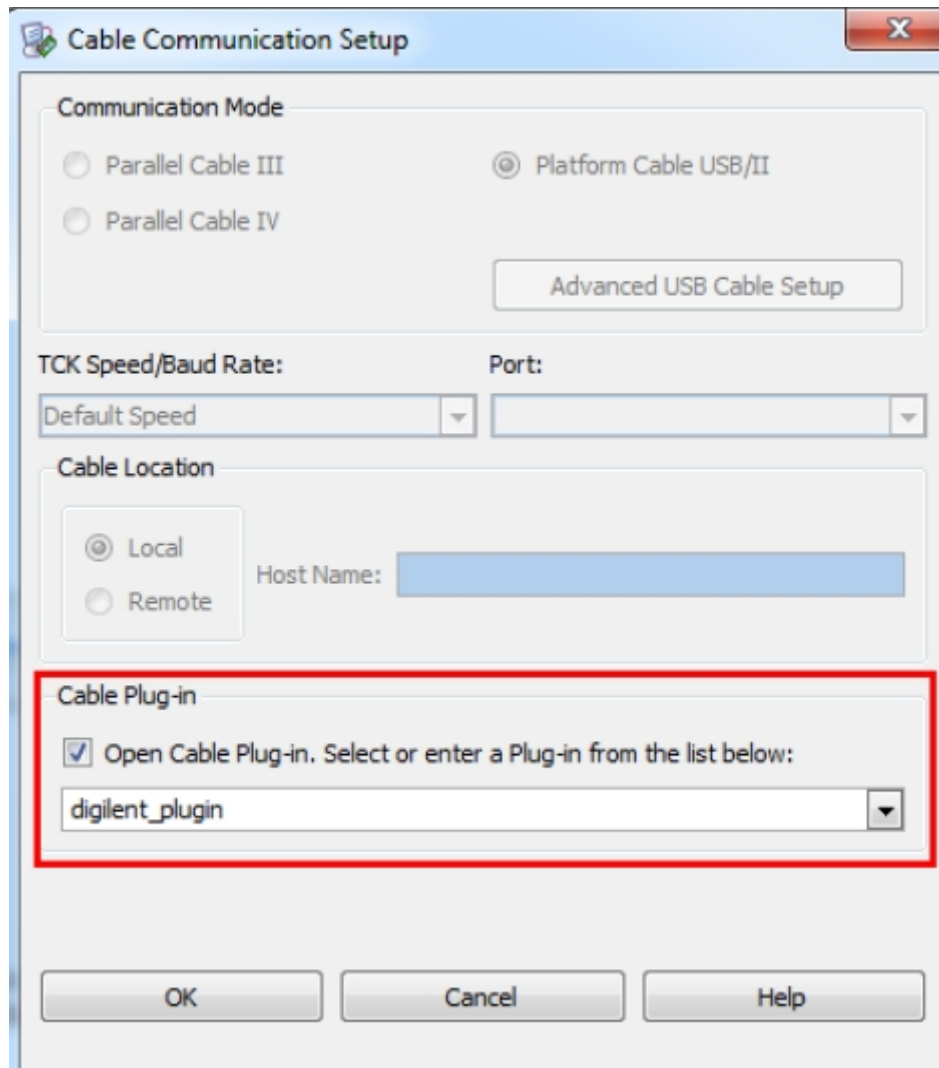
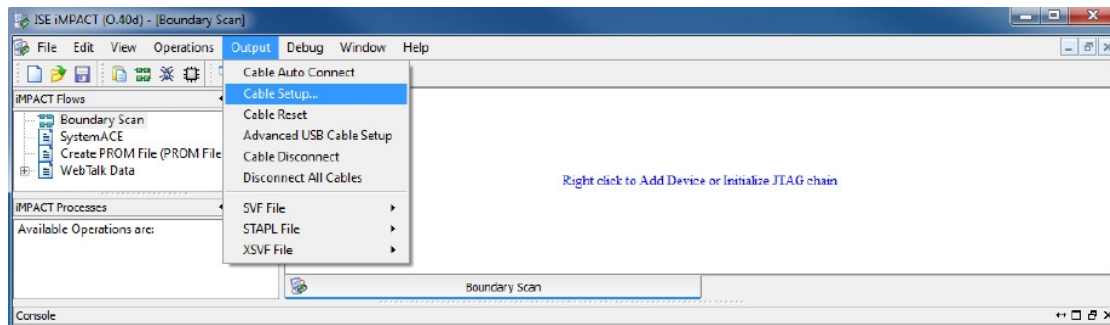
<http://www.digilentinc.com/Products/Detail.cfm?NavPath=2,66,768&Prod=DIGILENT-PLUGIN> 下载 digilent plugin，根据系统选择特定版本

按照软件包中的文档  Digilent Plug-in Xilinx v13.pdf 进行安装。基本上就两步，先是将两个文件拷贝到 ISE 安装目录的特定地方。

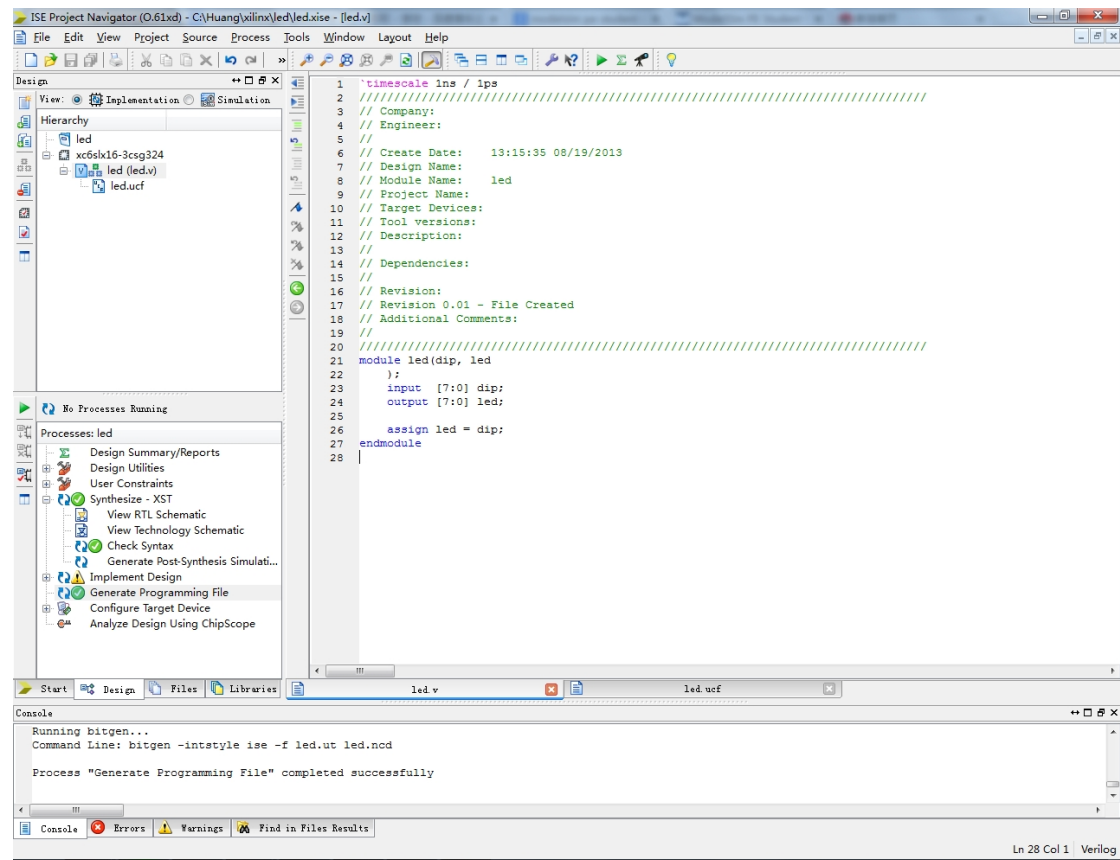
For the ISE Design Suite, the typical location is
C:\Xilinx\13.1\ISE_DS\ISE\lib\nt\plugins\Digilent\libCseDigilent
Note: For 64-bit Windows, use nt64 in place of nt



再就是在下载的时候设置下载线。



2 用户界面



由上到下主要分为标题栏、菜单栏、工具栏、工程管理区、源文件编辑区、过程管理区、信息显示区、状态栏等 8 部分

标题栏：主要显示当前工程的名称和当前打开的文件名称。

菜单栏：主要包括文件（File）、编辑（Edit）、视图（View）、工程（Project）、源文件（Source）、操作（Process）、窗口（Window）和帮助（Help）等 8 个下拉菜单。其使用方法和常用的 Windows 软件类似。

工具栏：主要包含了常用命令的快捷按钮。灵活运用工具栏可以极大地方便用户在 ISE 中的操作。在工程管理中，此工具栏的运用极为频繁。

工程管理区：提供了工程以及相关文件的显示和管理功能，主要包括源文件视图（Source View），快照视图（Snapshot View）和库视图（Library View）。其中源文件视图比较常用，显示了源文件的层次关系。快照是当前工程的备份，设计人员可以随时备份，也可以将当前工程随时恢复到某个备份状态。快照视图用于查看当前工程的快照。执行快照功能的方法是选择菜单项 Project | Take Snapshot。库视图则显示了工程中用户产生的库的内容。

源文件编辑区：源文件编辑区提供了源代码的编辑功能。

过程管理区：本窗口显示的内容取决于工程管理区中所选定的文件。相关操作和 FPGA 设计流程紧密相关，包括设计输入、综合、仿真、实现和生成配置文件等。对某个文件进行了相应的处理后，在处理步骤的前面会出现一个图标来表示该步骤的状态。

信息显示区：显示 ISE 中的处理信息，如操作步骤信息、警告信息和错误信息等。信息显示区的下方有两个标签，分别对应控制台信息区（Console）和文件查找区（Find in Files）。

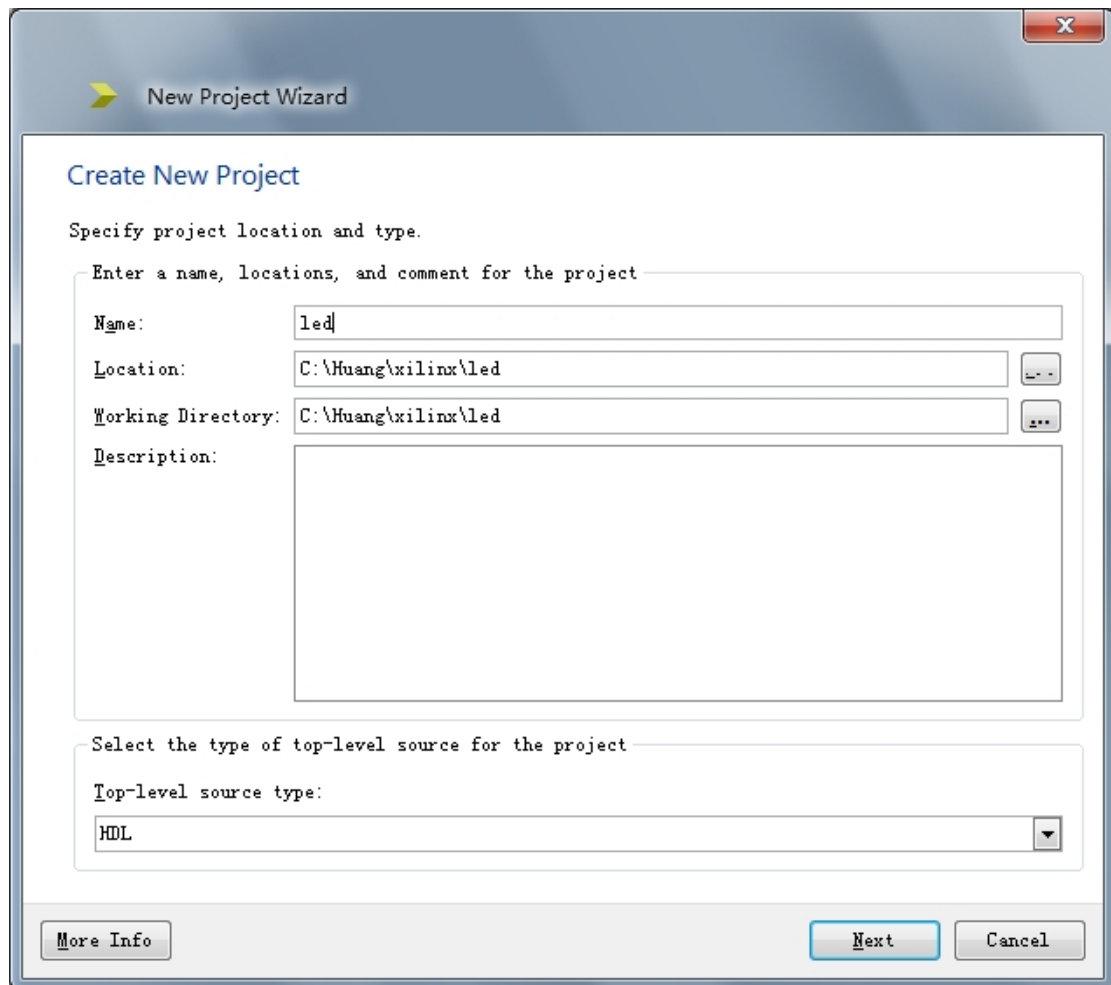
如果设计出现了警告和错误，双击信息显示区的警告和错误标志，就能自动切换到源代码出错的地方。

状态栏：显示相关命令和操作的信息。

3 基本使用-点亮开发板上的 LED 等

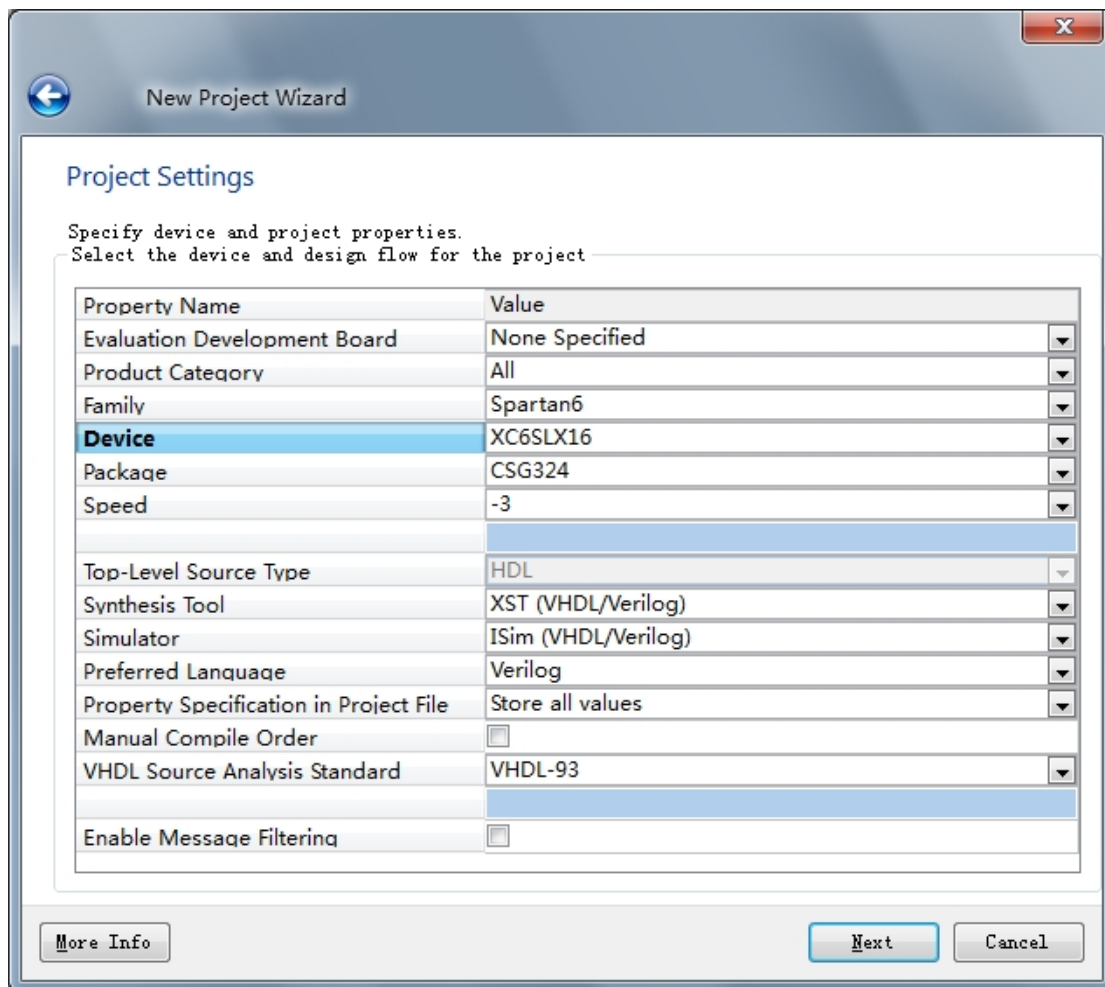
3.1 新建工程

点击 new project，为工程命名



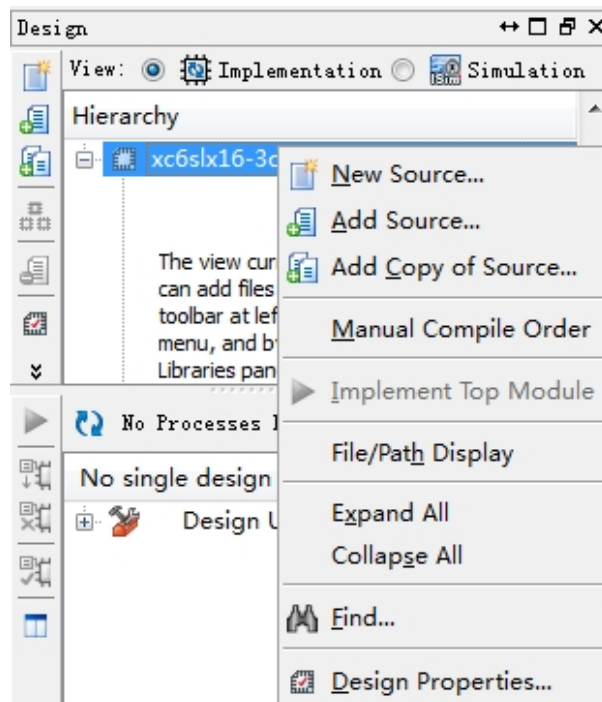
在工程设置中，选择合适的选项

- 1) 我们选用的开发版为 Spartan6 系列，XC6SLX16
- 2) Preferred language 选择 Verilog
- 3) 点击下一步
- 4) 完成工程设置

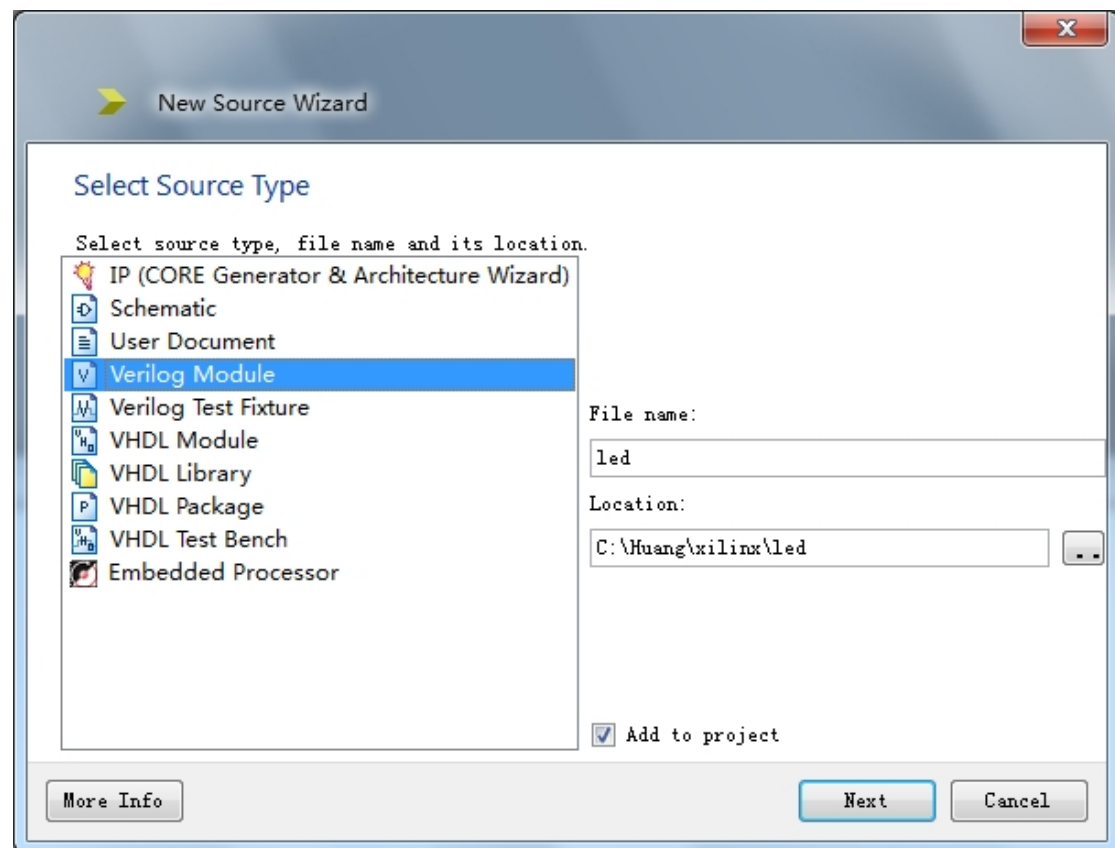


3.2 编写模块

在左上角工程右键点击我们创建的工程，选择 new source 创建新的源文件



选择新建 verilog 模块，并且命名



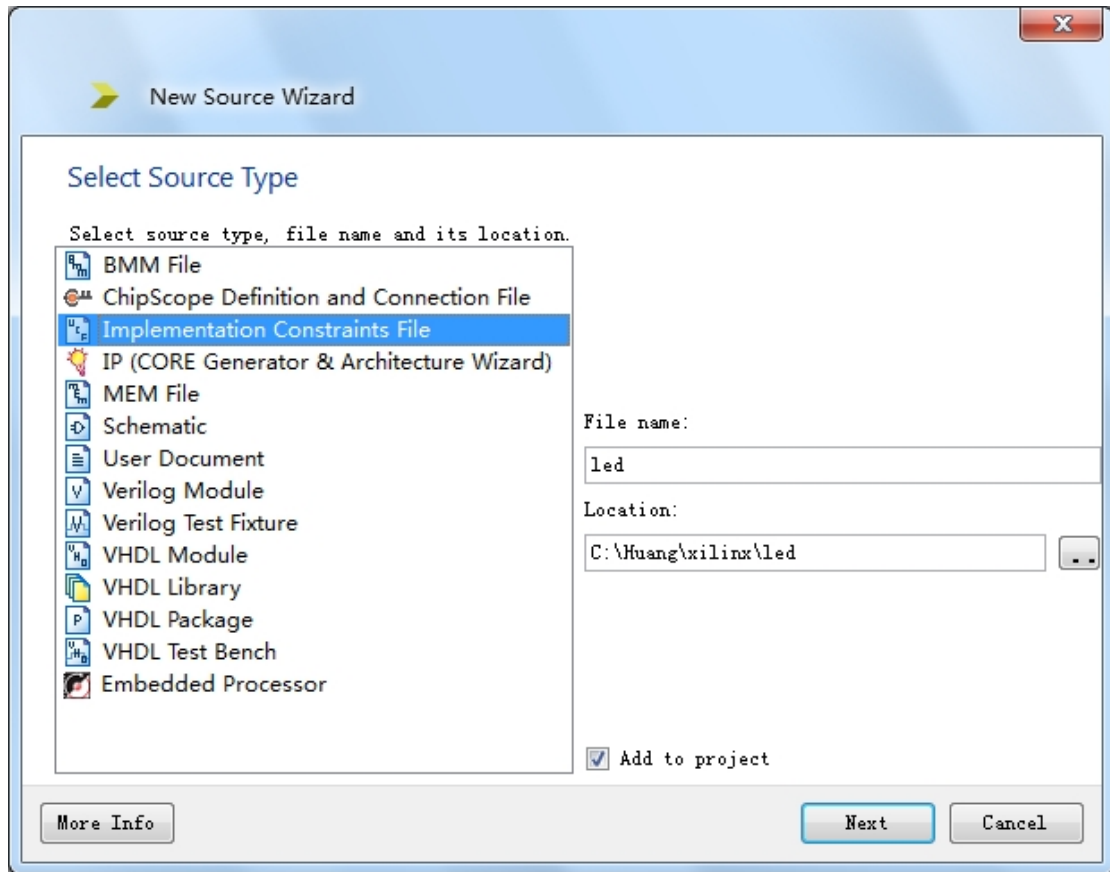
3.3 填入代码

```
20 //////////////////////////////////////////////////
21 module led(dip, led
22     );
23     input  [7:0] dip;
24     output [7:0] led;
25
26     assign led = dip;
27 endmodule
```

3.4 添加约束文件

在工程管理区的 view 中选择 Implementation，然后在过程管理区双击 Synthesize-XST，就可以开始综合过程

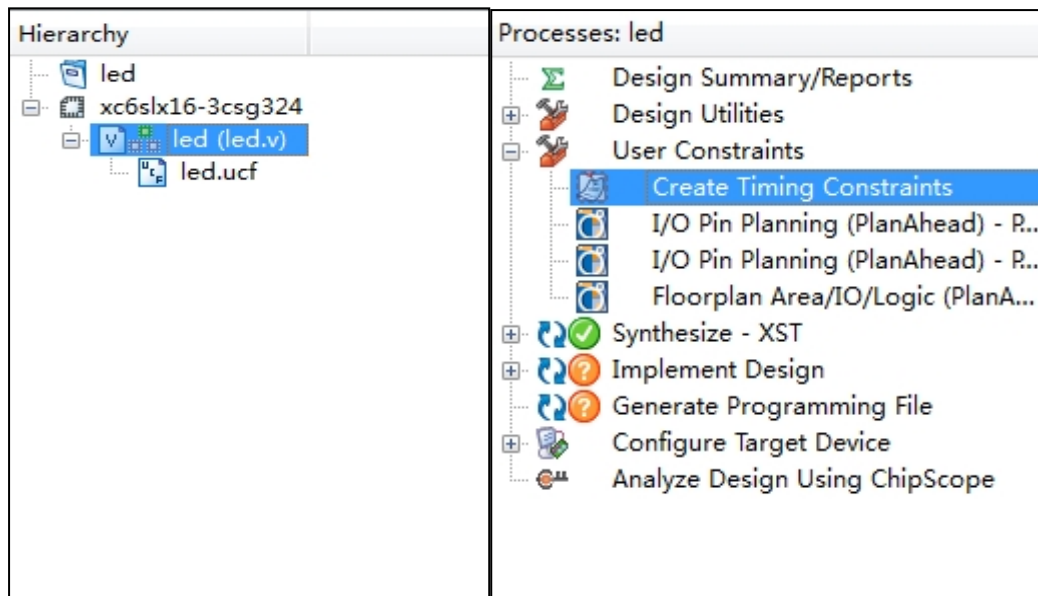
再次右击我们创建的工程，新建约束文件 led.ucf，确定模块端口与开发板引脚的关系，这次选择 Implementation Constraints Files，新建后编辑约束文件。



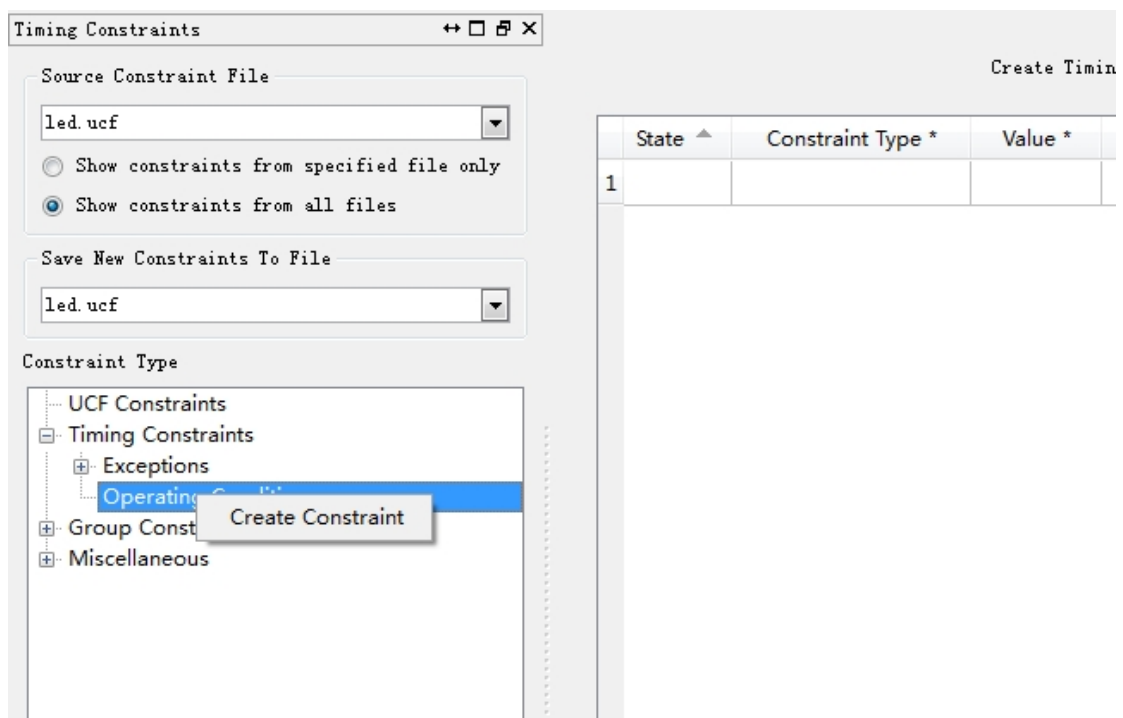
添加管脚约束，并保存。

```
1 Net dip<0> LOC = T10;
2 Net dip<1> LOC = T9;
3 Net dip<2> LOC = V9;
4 Net dip<3> LOC = M8;
5 Net dip<4> LOC = N8;
6 Net dip<5> LOC = U8;
7 Net dip<6> LOC = V8;
8 Net dip<7> LOC = T5;
9
10 Net led<0> LOC = U16;
11 Net led<1> LOC = V16;
12 Net led<2> LOC = U15;
13 Net led<3> LOC = V15;
14 Net led<4> LOC = M11;
15 Net led<5> LOC = N11;
16 Net led<6> LOC = R11;
17 Net led<7> LOC = T11;
```

另一种添加约束文件的方法是：在工程区选择顶端的模块，然后双击过程管理区中“User Constrains”下的“Create Timing Constrains”就可以打开“Constrains Editor”。

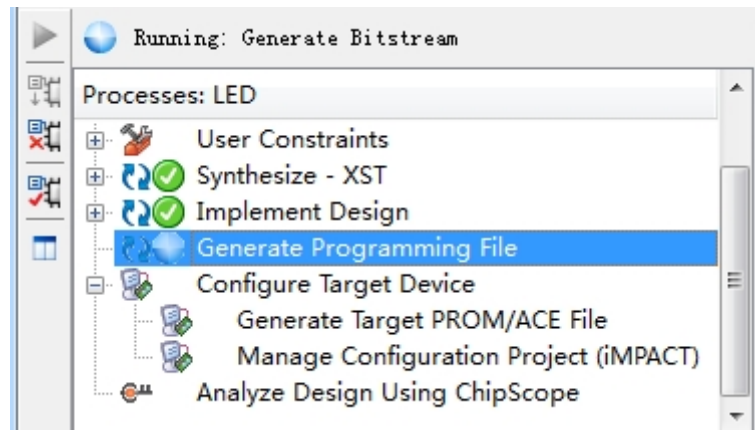


Constraints Editor 界面如下，可以通过选中具体的“Constraint Type”后右键，选择“Create Constraints”来编辑不同类型的约束条件。



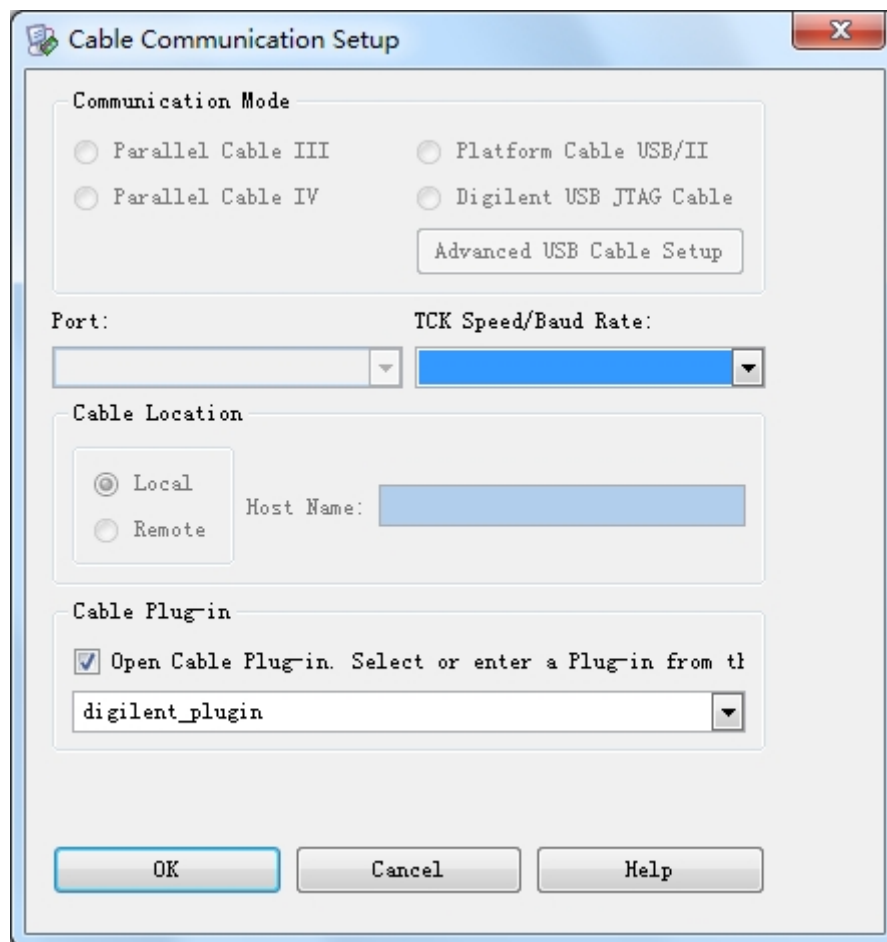
3.5 生成配置文件并对 FPGA 进行配置

双击左下角 Generate Programming File，成功后前面的图标会变成绿色的勾，并且在工程目录下生成一个*.bit 文件



双击 Configure Target Device，启动 ISE iMPACT

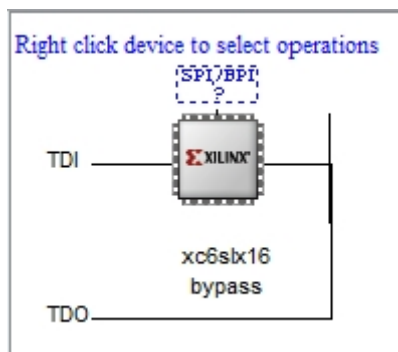
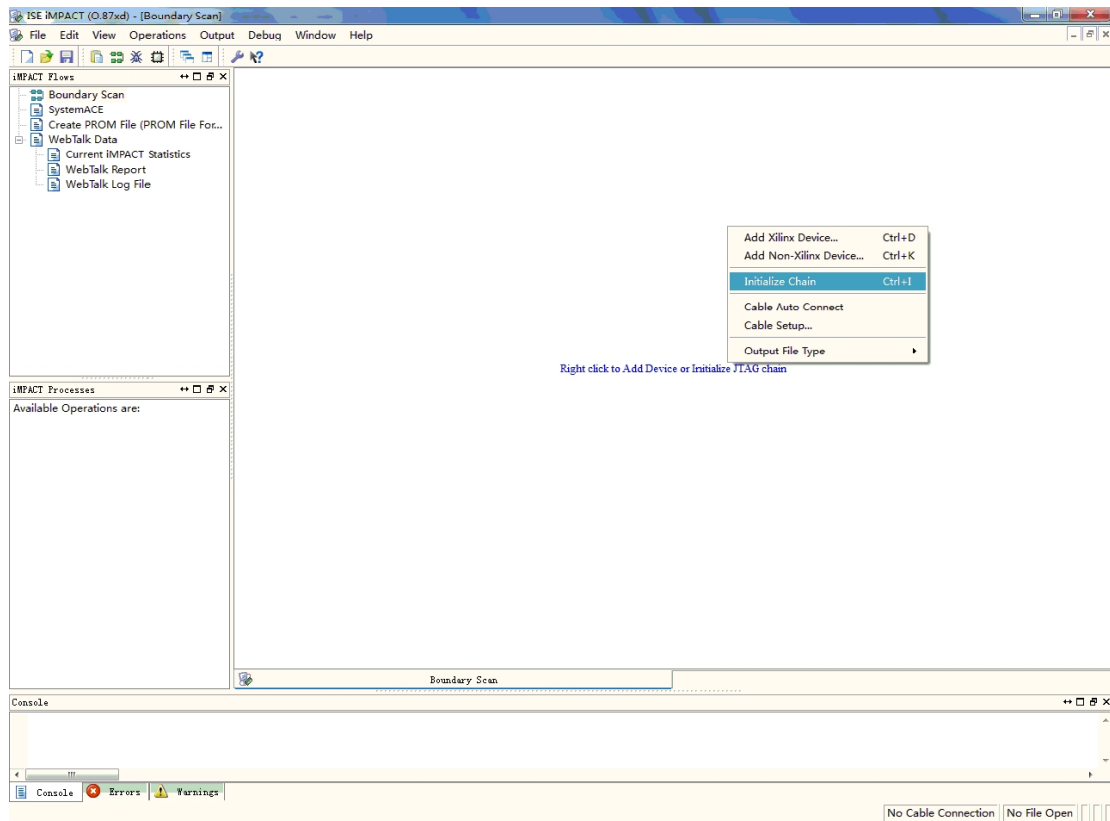
Output->Cable Setup...设置下载线



连接开发板的电源线，打开开发板电源

点击 Boundary Scan，在主界面区域中右键点击，并选择 Initialize Chain，找到 FPGA 设

备



右键点击 FPGA 图标，选择 assign new configuration file，找到工程目录下的.bit 文件
 右键点击 FPGA 图标，点击 program，成功后，调节板上的拨动开关，对应的 LED 灯
 会熄灭或点亮

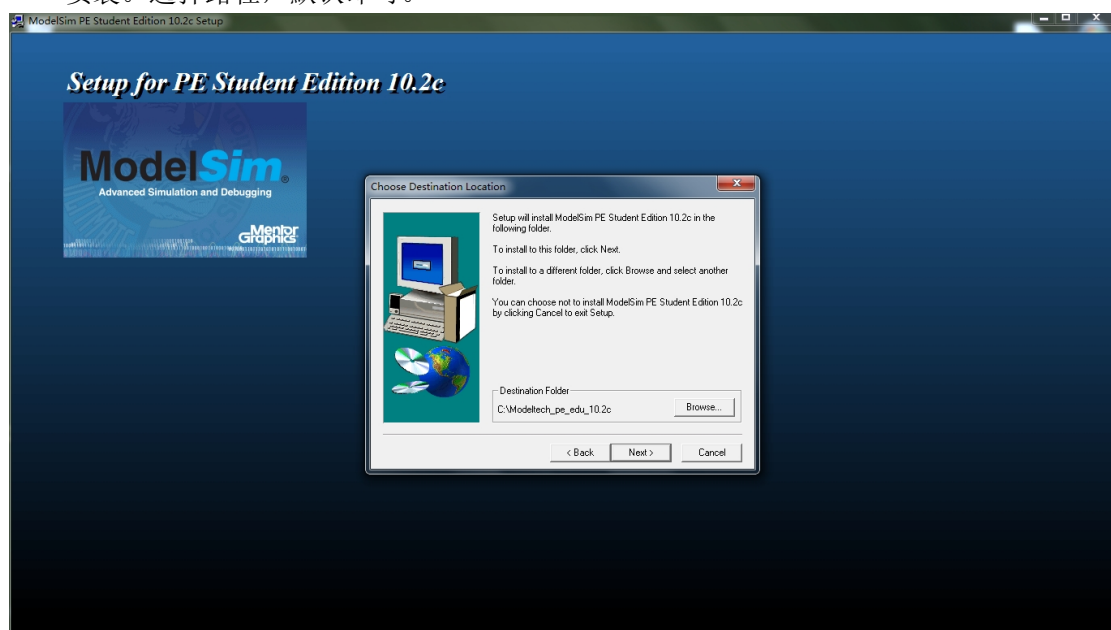
第四章 Modelsim

Modelsim 仿真软件是由 MentorGraphic 公司的子公司 Model 技术公司开发的工业界上最为通用的仿真器之一。它可以用于 Verilog 仿真、VHDL 仿真或者两者的混合仿真。

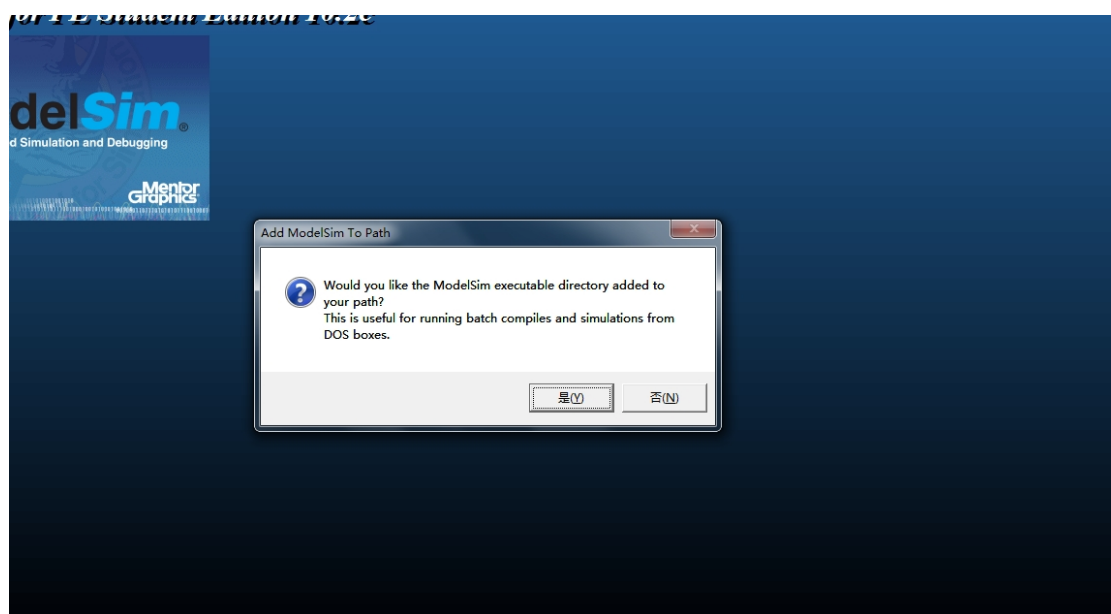
1 下载安装

到 http://www.mentor.com/company/higher_ed/modelsim-student-edition 下载 ModelSim PE Student Edition。

安装。选择路径，默认即可。

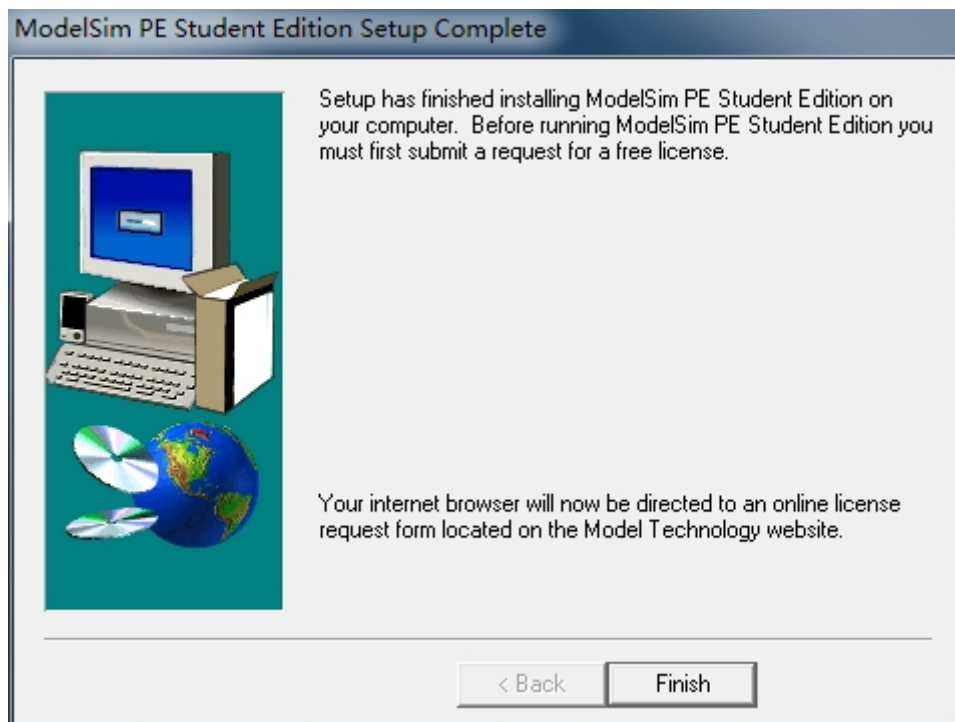


选择“是”，添加系统环境变量。

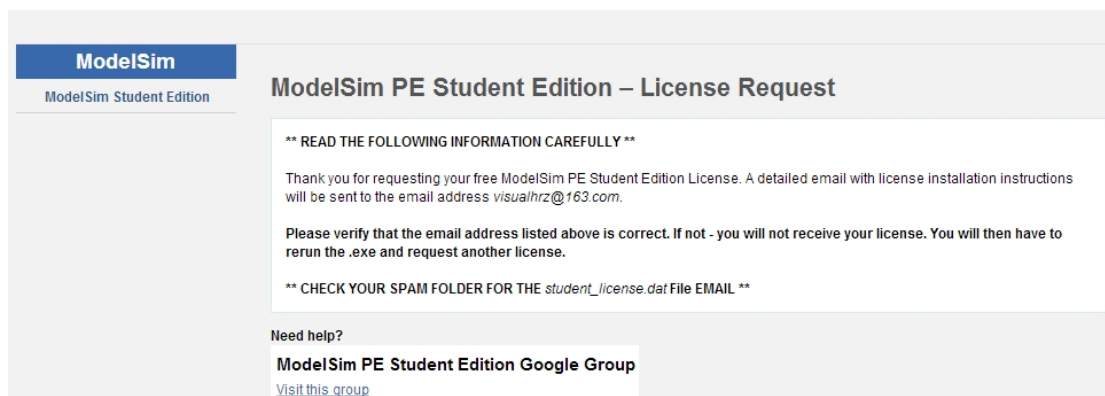


完成。根据提示去获取 license。也可直接去访问如下链接获取 license。

http://portal.model.com/d/license_request.asp



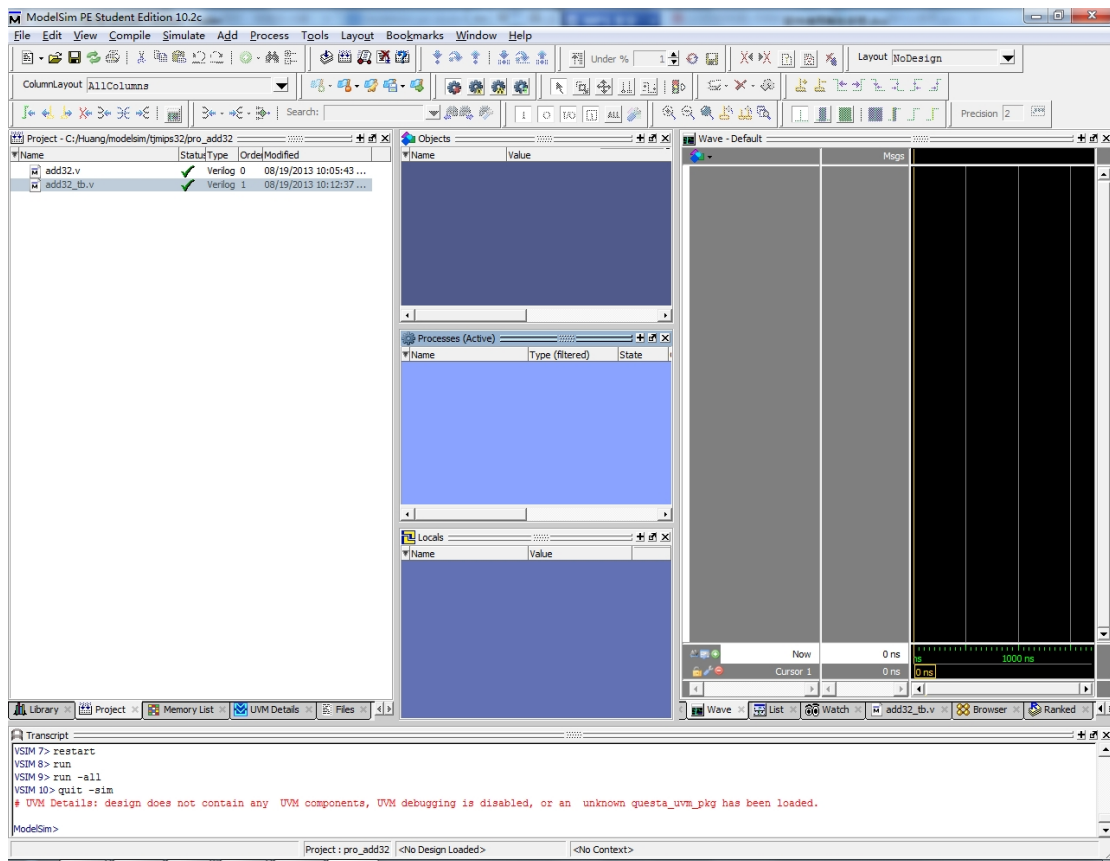
填写好信息，到登记邮箱去下载 license。



将下载的 student_license.dat 拷贝到 modelsim 安装路径的顶层目录下，如 C:\Modeltech_pe_edu_10.2c。不要修改 student_license.dat。

2 用户界面

2.1 主窗口



2.2 library 窗口

呈现编译好的库文件

2.3 project 窗口

呈现当前项目文件

2.4 sim 窗口

仿真时自动打开，呈现仿真内容

2.5 objects 窗口

Objects 窗口显示了被选中的设计层次模块的信号名以及它们的值。信号可以是VHDL信号，可以是Verilog 线网、寄存器变量和命名事件。窗口中的信号能够支持“拖放”功能，即将信号拖放到Wave 窗或者List 窗。Edit 菜单中的Force 菜单可以用于产生激励。View 菜单中的Filter 可以帮助快速显示或者不显示想要观察的信号类型，例如输入，输出，或者内部信号等。

2.6 process 窗口

2.7 locals 窗口

Locals 窗口显示下面即将被执行的语句中的可以马上被看到的数据对象及其值。（即将被执行的语句在源程序中是被绿色的箭头标示的）。Locals 窗口包含了两列，第一列列出了数据对象的名字，第二列是其数值。

2.8 watch 窗口

Watch 窗口显示了当前仿真时间下的信号与变量的值，与Objects 窗口和Locals 窗口不同的是，Watch 窗口允许你观察当前设计中任意的信号与变量的值。

在 Watch 窗口中被观察的值可以是VHDL 的信号、变量、generic，也可以是Verilog 的线网、寄存器、命名事件与模块参数。要在Watch 窗口观察信号或者变量，只需要从Object

窗口或者Locals 窗口中将信号拖放进来即可。

2.9 wave 窗口

显示待观察信号的波形

3 基本使用

ModelSim 有三种实现方法。第一种是交互式的命令行，这种操作方法没有用户界面，唯一的界面是控制台的命令行。第二种是用户界面(UI)，它能够接收菜单输入和命令行输入。第三种是批处理模式，是用DOS 或UNIX 命令行运行批处理文件。我们在这里主要讨论第二种——用户界面的方式。

3.1 建立 Modelsim 库

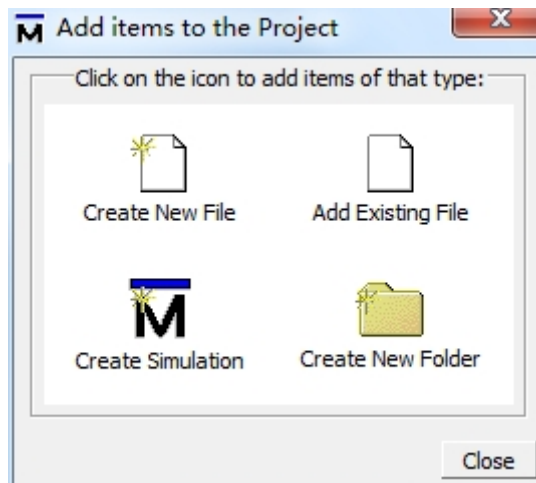
从主菜单里面：File->New->Library 点击Library 得到Create a New Library 对话框。选择a new library and a logical mapping to it，在Library Name 中输入work，相应的在Library Physical Name 中也会出现物理名work。然后点击OK 确定。

此时在工作空间 workspace 的Library 栏内会出现一个名为work 的库。

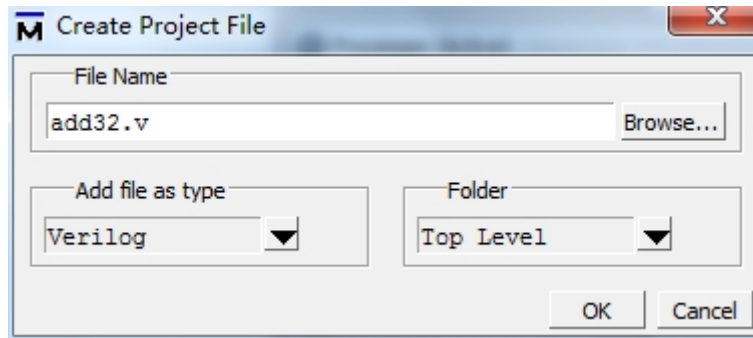
3.2 建立项目

从主菜单里面：File->New->Project 点击Project 得到Create Project 窗口，在Project Name栏输入工程名pro_add32，在Project Location 栏输入新建工程所处的位置（默认为用户当前所在的工作目录下），在Default Library Name 栏中输入所使用的库名（默认为work 库）。输入完毕以后点击OK 确认。

此时在工作空间 workspace 内会出现一个名为Project 的栏。同时出现了一个Add items to the Project 的对话框。如果源文件已经存在，选择Add Existing File 后又会弹出一个Add file to Project 的对话框，将需要的源文件添加到当前项目即可。



否则，新建源文件 add32.v，文件类型为 Verilog，添加如下代码。



```
1
2  module add32(a, b, result);
3      input  [31:0] a, b;
4      output [31:0] result;
5
6      assign result = a + b;
7  endmodule
```

同样，新建源文件 add32_tb.v，文件类型为 Verilog，添加如下代码。

```
1  module add32_tb();
2      reg  [31:0] a, b;
3      wire [31:0] result;
4
5      add32 adder(a, b, result);
6
7      initial
8      begin
9          a = 0; b = 0;
10         #100;
11         a = 1; b = 8;
12         #100;
13         a = 32'h12345678;
14         b = 32'h87654321;
15         #100;
16         b = 0;
17     end
18
19 endmodule
```

编辑好后，在Project 栏中出现add32.v、add32_tb.v 两个文件。同时它们的状态Status 被显示为?，代表了这两个文件现在还没有被编译。

3.3 编译源代码

点击右键在下拉菜单中点击Compile->Compile All，此时两个文件都会被编译。也可以分别选中两个文件右键选择Compile Selected 对两个文件分别编译。

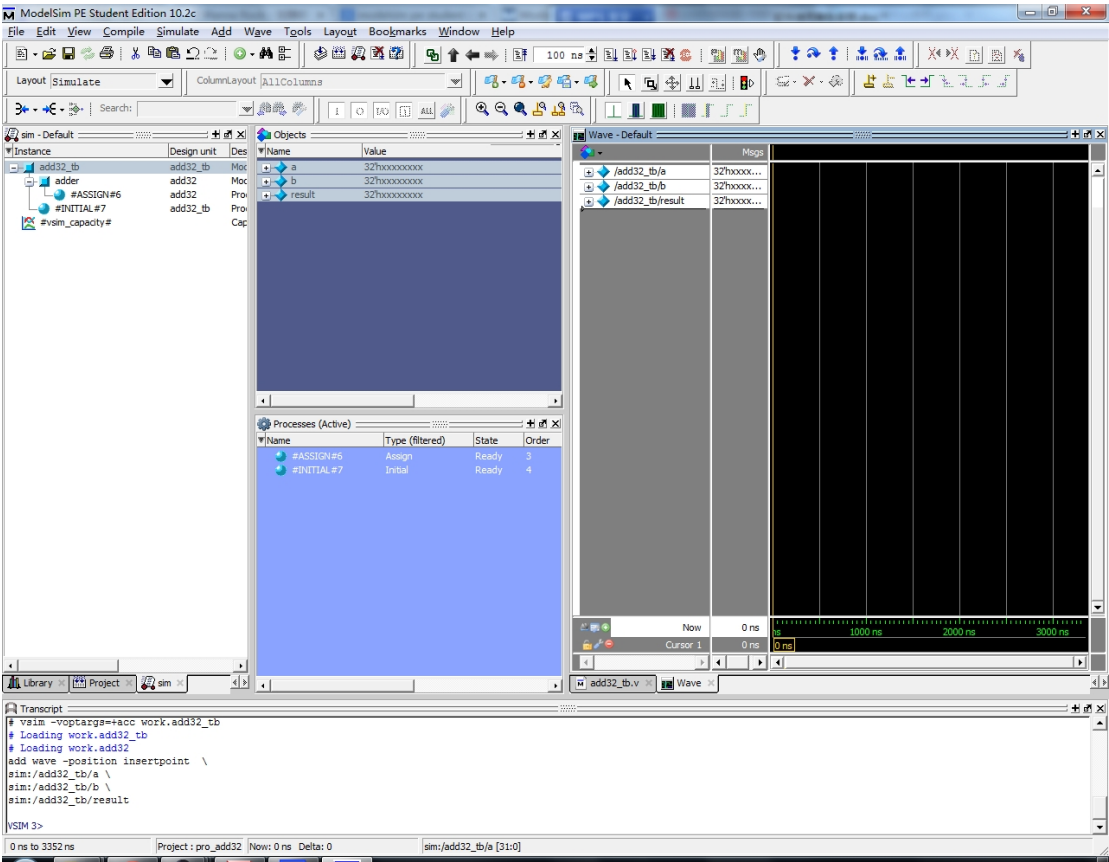
编译完以后用户界面如图2.7 所示：此时状态Status 显示为绿色的对号(√)表示编译已通过。

3.4 启动仿真器

编译通过以后，点击 Library 栏，展开work库。发现此时在work 库内部出现了add32 与add32_tb两个模块(图标表示是Verilog 的module)。选择测试模块tb_nand2（注意：必须选

择顶层的测试模块) 点击右键得到下拉菜单, 选择Simulate, 就可以加载要仿真的设计了。

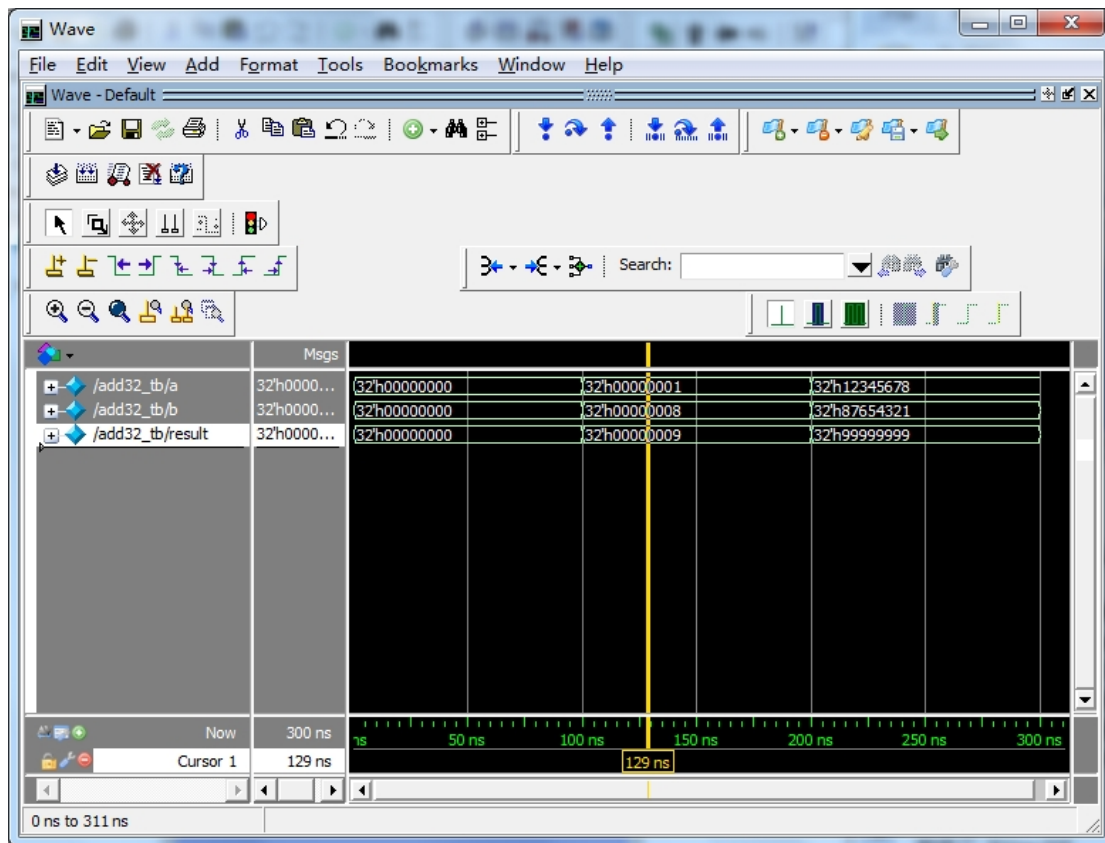
点击 Simulate后在Project栏旁边出现sim 栏。选择sim栏中不同的实例, Objects栏中的信号会发生变化。点击add32_tb, 出现下拉菜单, 选择Add->Add to Wave。此时用户界面出现 Wave-default 新的内嵌窗口, 波形窗中出现了待观察的信号a、b、result。



3.5 运行仿真器

在 Transcript 窗口中输入run -all, 然后enter 键确定(或点击工具栏按钮), 运行仿真。在仿真结束时弹出 Finish Vsim 窗口, 选择No, 否则会退出仿真器。波形窗内嵌在主窗口内很不容易观察, 可以点击图2.14 中波形窗右上角的按钮 (Dock/Undock)将波形窗单独调出来观察。

调出来的波形窗口如图 2.15 所示, 可以通过拉动波形窗下面的滚卷条来调节显示波形的的位置时刻, 点击工具栏中的按钮可以从整体上对波形进行观察。



观察波形，如果波形是错误的，可以在用户工作区修改源代码。修改完代码以后保存，重新运行仿真。重新运行仿真时点击工具栏中的Restart 按钮，弹出Restart 对话框如图2.16所示。

点击 Restart 按钮以后，重新点击按钮执行仿真。

如果观察波形正确，说明设计达到了要求，可以退出仿真器。首先在 Transcript 窗口中输入命令quit -sim 后enter 键确定执行(也可以利用菜单Simulate->End Simulation 菜单完成，不过此时要在弹出的对话框中选中YES 确定退出仿真)，退出仿真，如图2.17 所示。

利用 quit -sim 命令退出仿真以后的界面又恢复到了启动仿真前的状态，如图2.18 所示。注意右边用户界面处的几个窗口同学们可以自己关掉。

3.6 调试

可以采用两种方式，一是信号监视，将要监视的信号拖拽到wave窗口；二是设置断点。可以在源代码窗口设置断点。还可以设置条件断点。

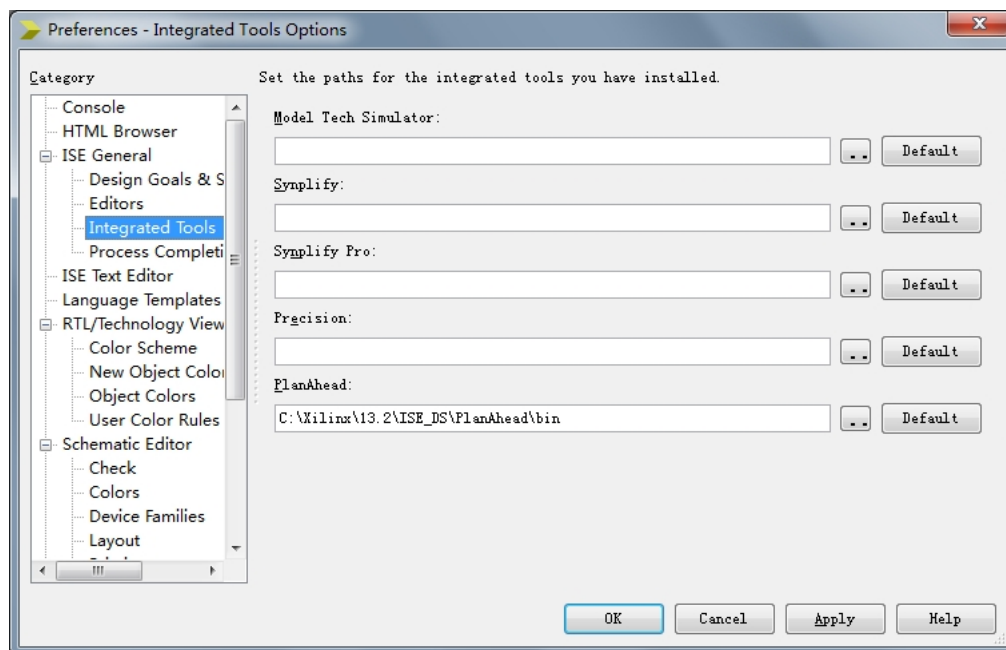
```
1  module add32_tb();
2      reg  [31:0] a, b;
3      wire [31:0] result;
4
5      add32 adder(a, b, result);
6
7      initial
8      begin
9          a = 0; b = 0;
10         #100;
11         a = 1; b = 8;
12         #100;
13         a = 32'h12345678;
14         b = 32'h87654321;
15         #100;
16         b = 0;
17     end
18
19 endmodule
```

第五章 ISE 与 Modelsim 关联

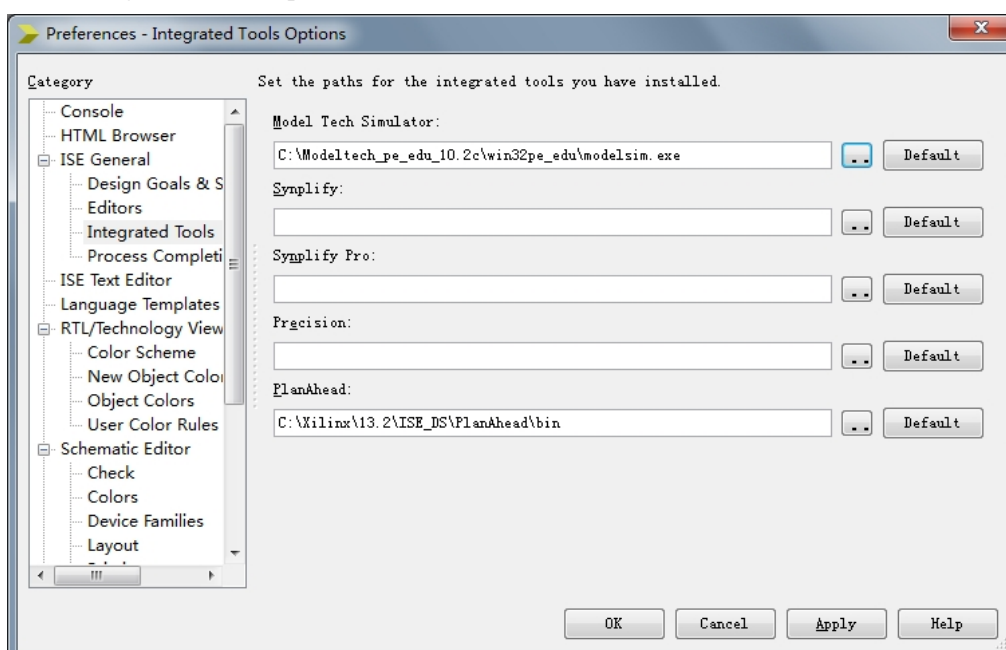
1 关联设置

目标：在 ISE 中直接调用 Modelsim 对设计进行仿真。虽然 ISE 自带仿真工具 ISim，但是 Modelsim 更为专业，功能更强大，更方便稳定，用户熟悉度高，希望大家使用 Modelsim 进行仿真。前提条件：先将 modelsim 与 ise 软件安装好。

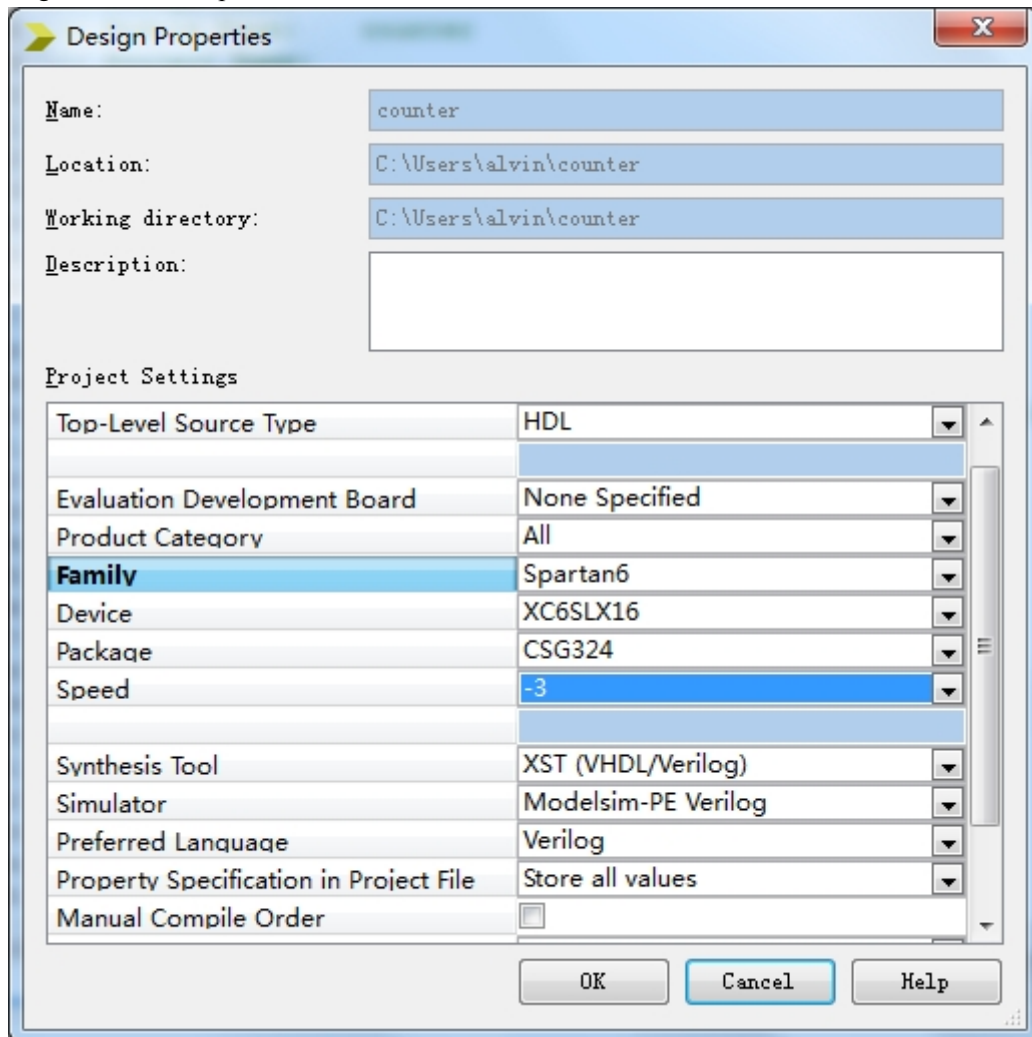
Edit->Preferences，打开 preferences 设置对话框。



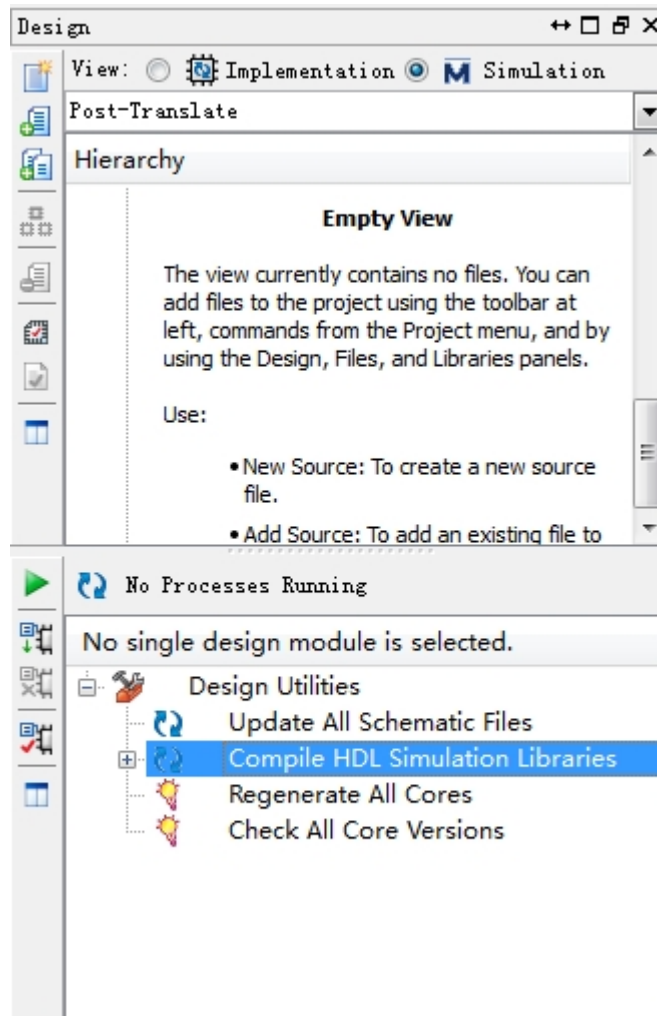
选择 Integrated Tools Options，将 Modelsim 的可执行文件导入到 Model Tech Simulator。



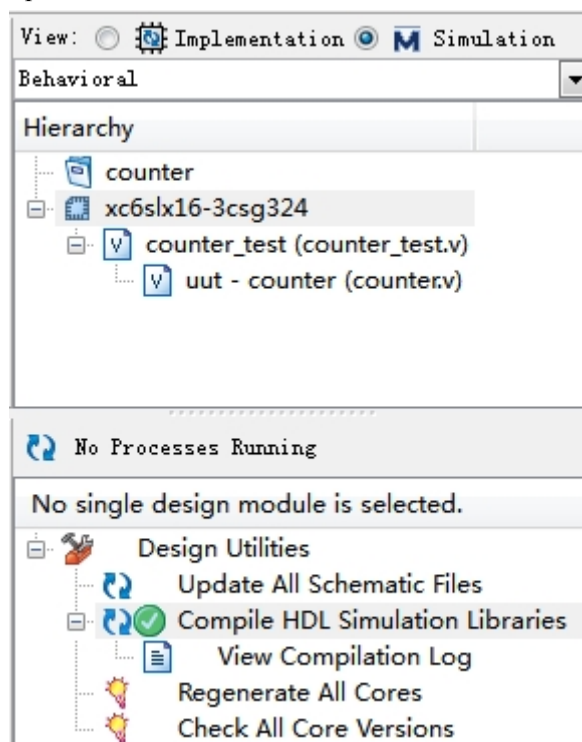
打开一个 Example Project。右键点击顶层设计，Design Properties，将 Simulator 改为 Modelsim-PE Verilog。将 FPGA 器件参数改成下图。Family: Spartan6，Device: XC6SLX16，Package: CSG324，Speed: -3。



在两个 Radio Button 中选择 Simulation。双击操作栏中的 Compile HDL Simulation Libraries。



编译完成后，Compile HDL Simulation Libraries 前变成绿色勾状。



默认的库编译路径在 C:\Xilinx\13.2\ISE_DS\ISE\verilog\mti_pe\10.2c\nt(64)。几个文件夹代表编译好的库。

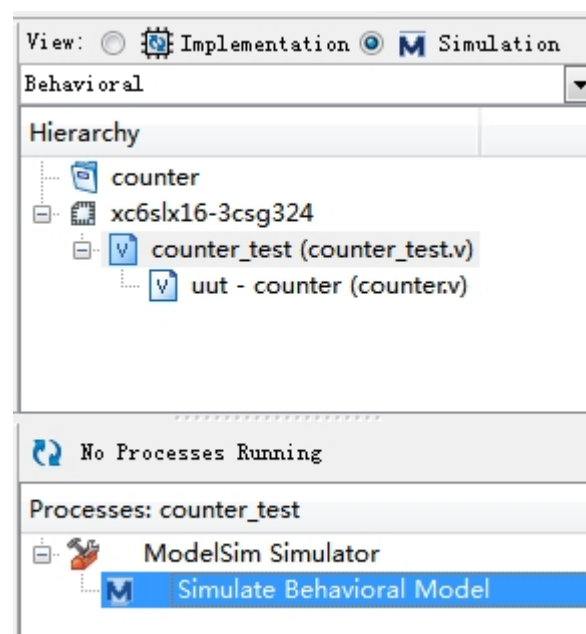
	secureip	2013/9/17 23:57	文件夹
	simprims_ver	2013/9/17 23:57	文件夹
	unimacro_ver	2013/9/16 20:12	文件夹
	unisims_ver	2013/9/17 23:57	文件夹
	xilinxcorelib_ver	2013/9/17 23:57	文件夹

将 C:\Modeltech_pe_edu_10.2c\modelsim.ini 的只读属性去掉，编辑 modelsim.ini，在其中的[library]后面加入下面蓝底文字。C:\Xilinx\13.2\ISE_DS\ISE\verilog\mti_pe\10.2c\nt(64)为编译库的默认存放路径。注意 32 位和 64 位系统。

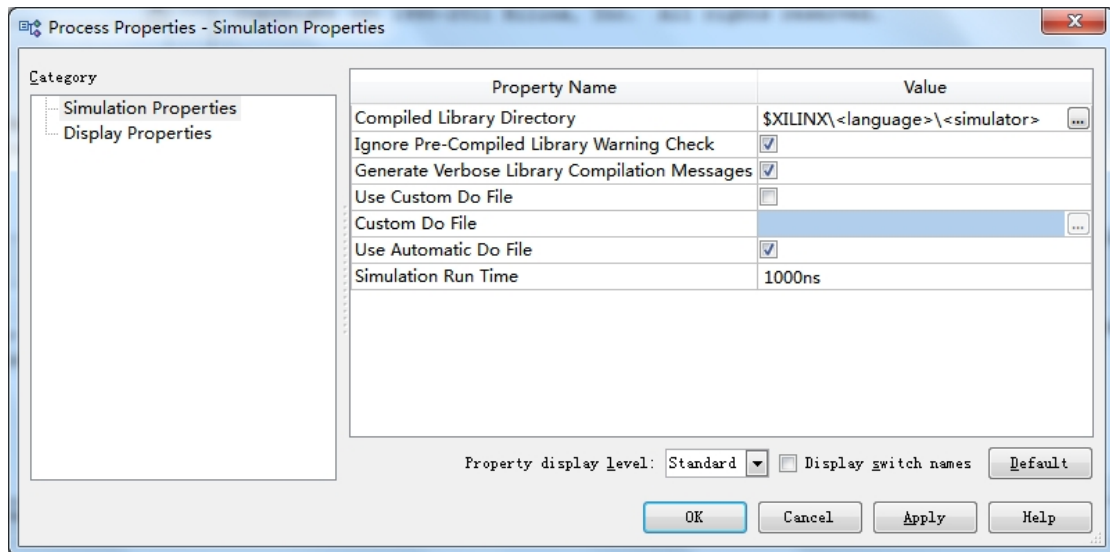
```
[Library]
std = $MODEL_TECH/./std
ieee = $MODEL_TECH/./ieee
vital2000 = $MODEL_TECH/./vital2000

secureip = C:\Xilinx\13.2\ISE_DS\ISE\verilog\mti_pe\10.2c\nt\secureip
simprims_ver = C:\Xilinx\13.2\ISE_DS\ISE\verilog\mti_pe\10.2c\nt\simprims_ver
unimacro_ver = C:\Xilinx\13.2\ISE_DS\ISE\verilog\mti_pe\10.2c\nt\unimacro_ver
unisims_ver = C:\Xilinx\13.2\ISE_DS\ISE\verilog\mti_pe\10.2c\nt\unisims_ver
xilinxcorelib = C:\Xilinx\13.2\ISE_DS\ISE\verilog\mti_pe\10.2c\nt\xilinxcorelib
```

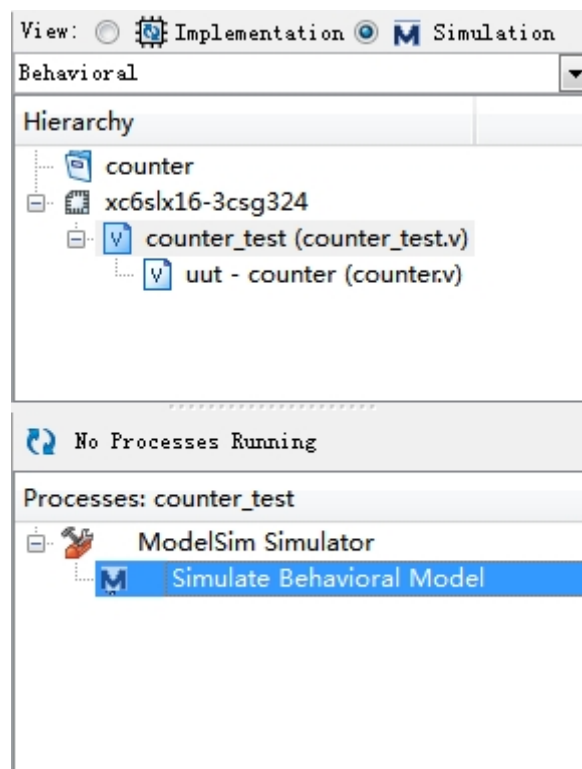
选中顶层设计，右键 Simulate Behavioral Model->Process Properties



设置为 Default，将 Ignore Pre-Compiled Library Warning Check 勾上。

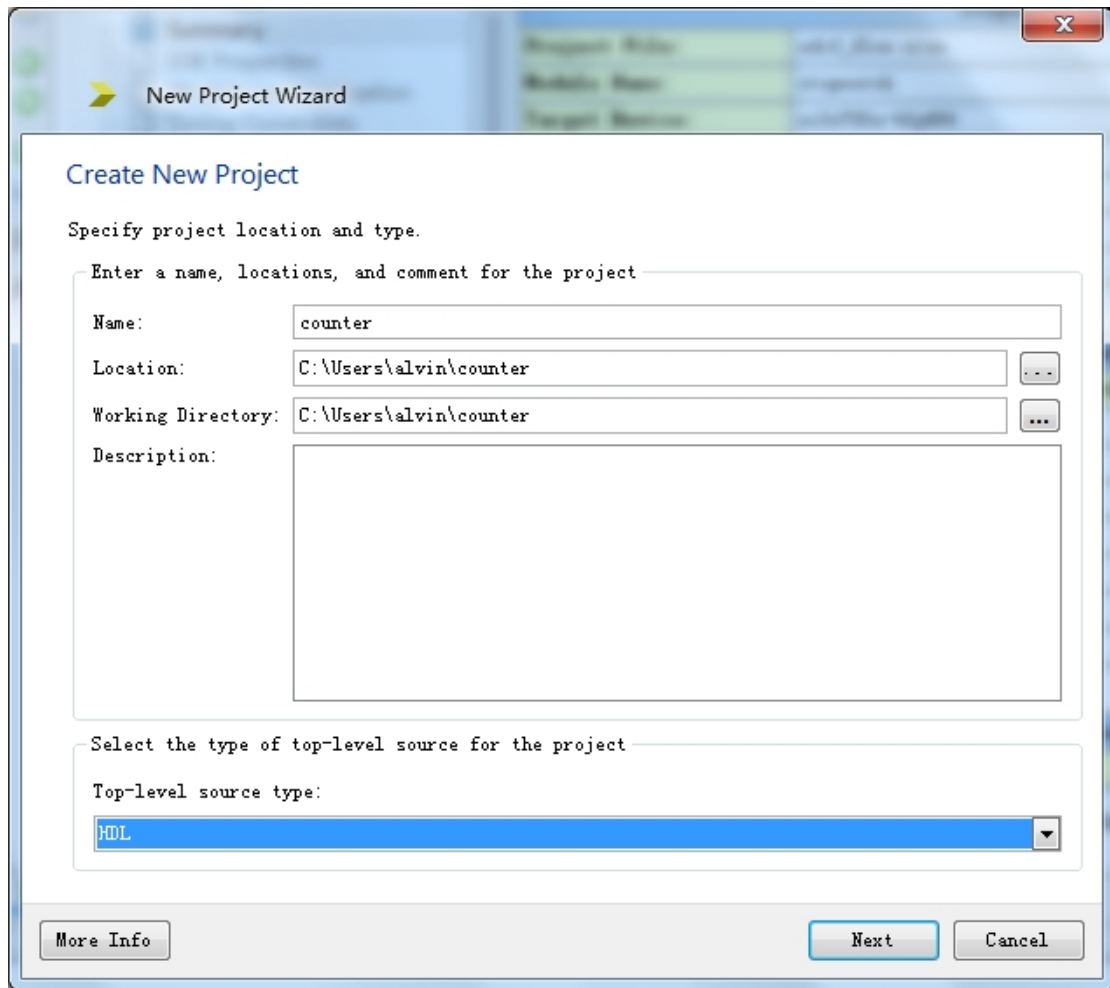


选中顶层设计，在操作栏双击 Simulate Behavioral Model 即可调用 Modelsim 进行仿真。



2 基本使用-4 位同步复位计数器

新建项目。File->New Project, Name 为 counter, Top-level source type 为 HDL。



注意 Simulator 为 Modelsim-PE Verilog。

Design Properties

Name: counter

Location: C:\Users\alvin\counter

Working directory: C:\Users\alvin\counter

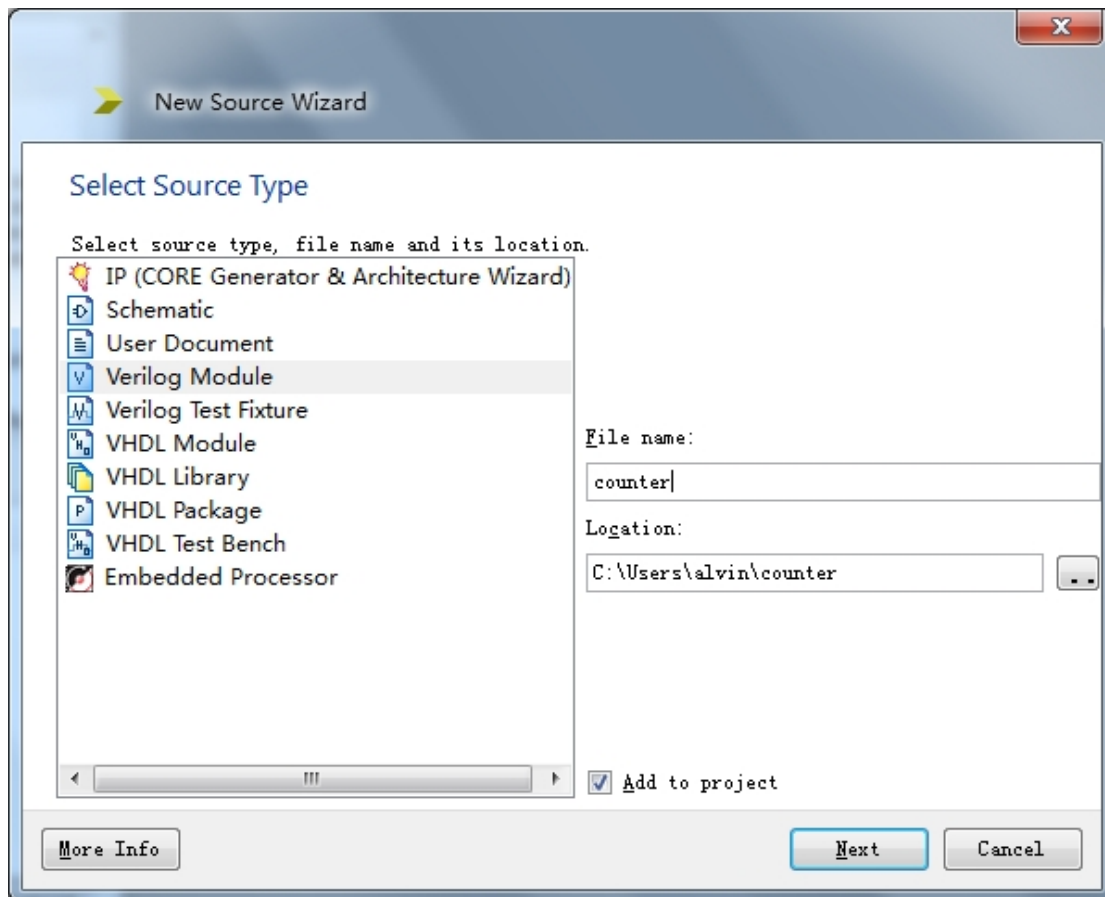
Description:

Project Settings

Property Name	Value
Top-Level Source Type	HDL
Evaluation Development Board	None Specified
Product Category	All
Family	Spartan6
Device	XC6SLX16
Package	CSG324
Speed	-3
Synthesis Tool	XST (VHDL/Verilog)
Simulator	Modelsim-PE Verilog
Preferred Language	Verilog
Property Specification in Project File	Store all values

OK Cancel Help

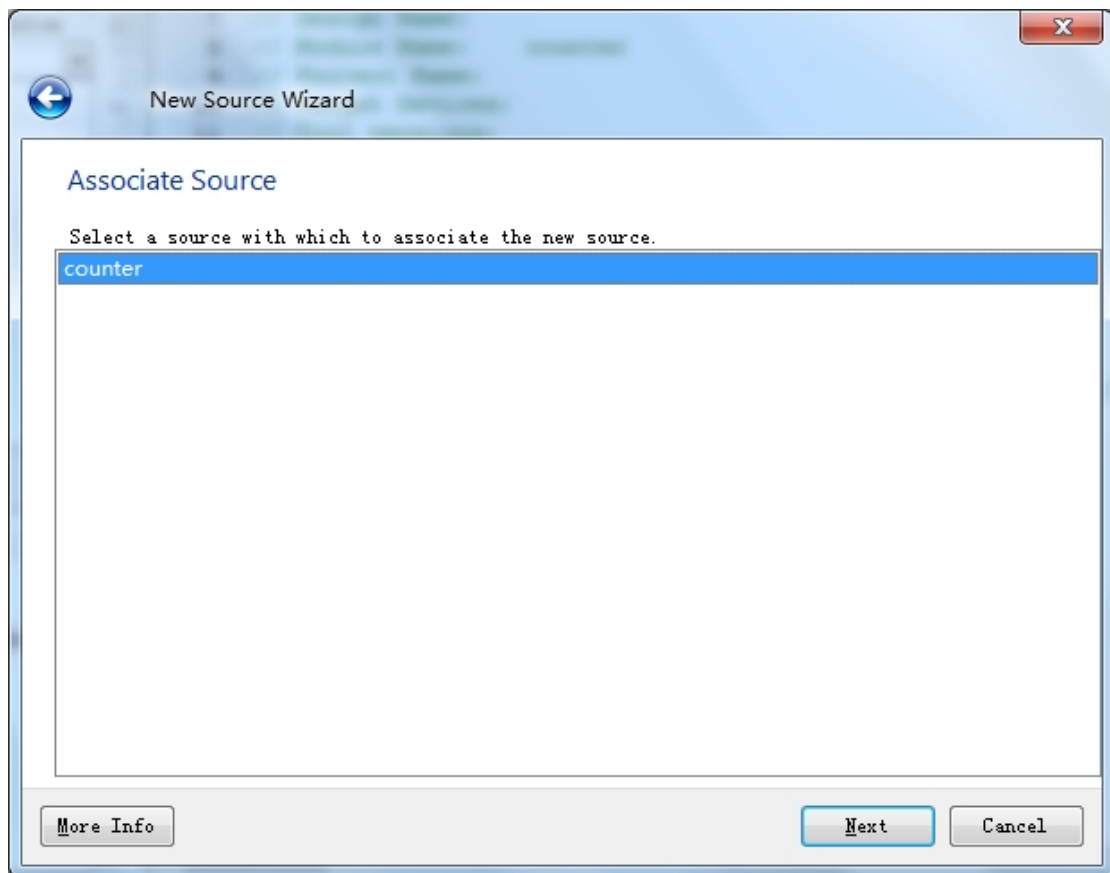
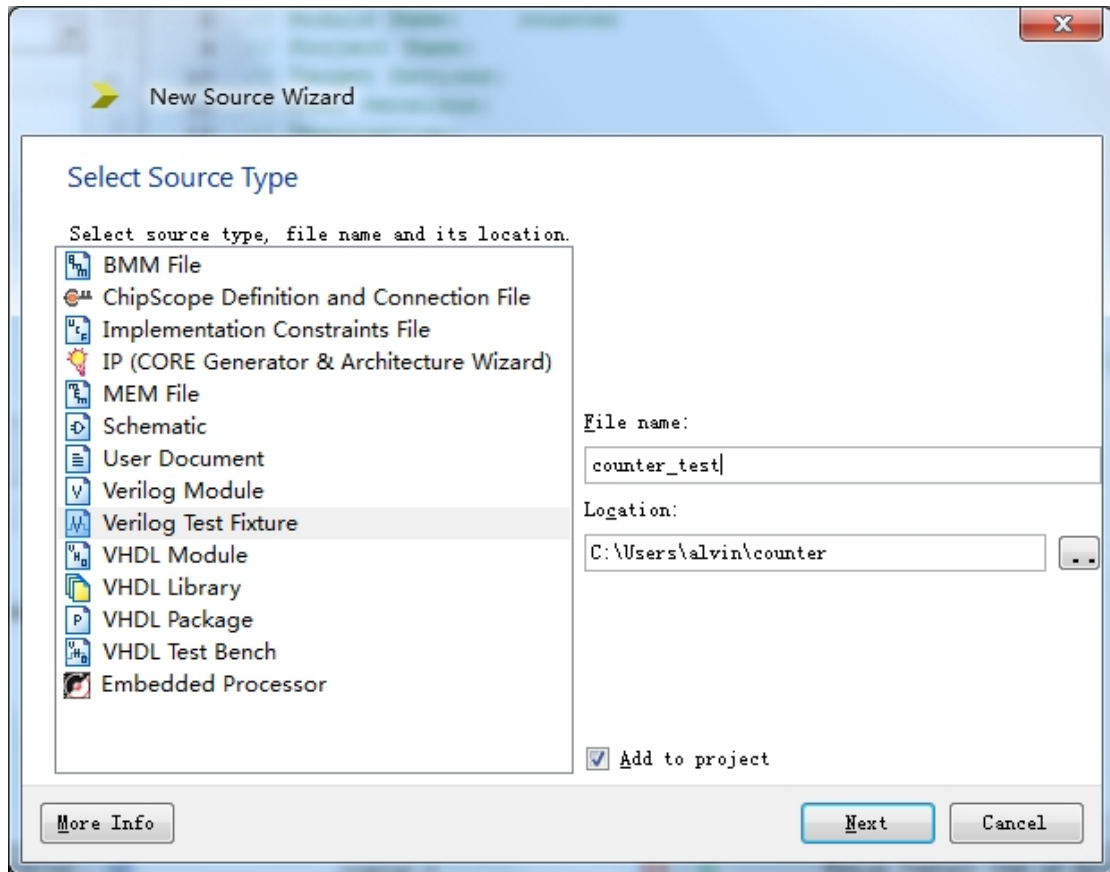
在设计栏中单击右键，选择 New source，选择 Verilog Module，输入文件名



在 counter.v 中输入代码。

```
20 //////////////////////////////////////////////////
21 module counter(out, reset, clk
22 );
23     output [3:0] out;
24     input      reset, clk;
25
26     reg    [3:0] out;
27
28     always@(posedge clk)
29     begin
30         if(reset) out <= 0;//reset
31         else      out <= out + 1;
32     end
33
34 endmodule
35
```

新建 counter.v 的 testbench 文件，该文件给 counter 设计提供测试激励。



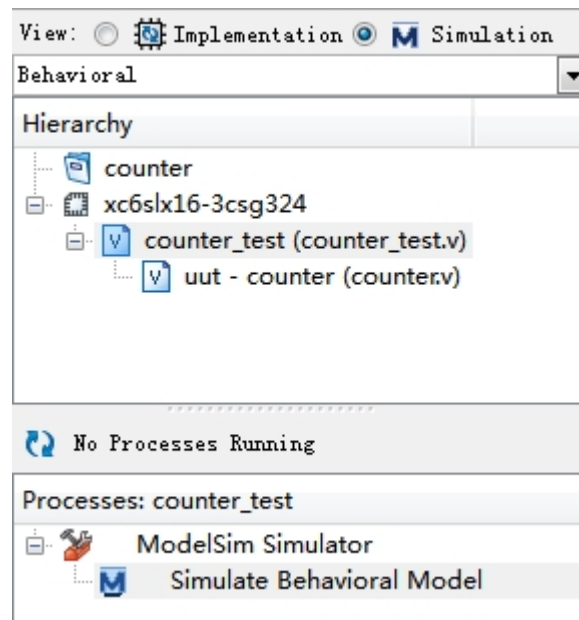
编辑 counter_test.v。

```

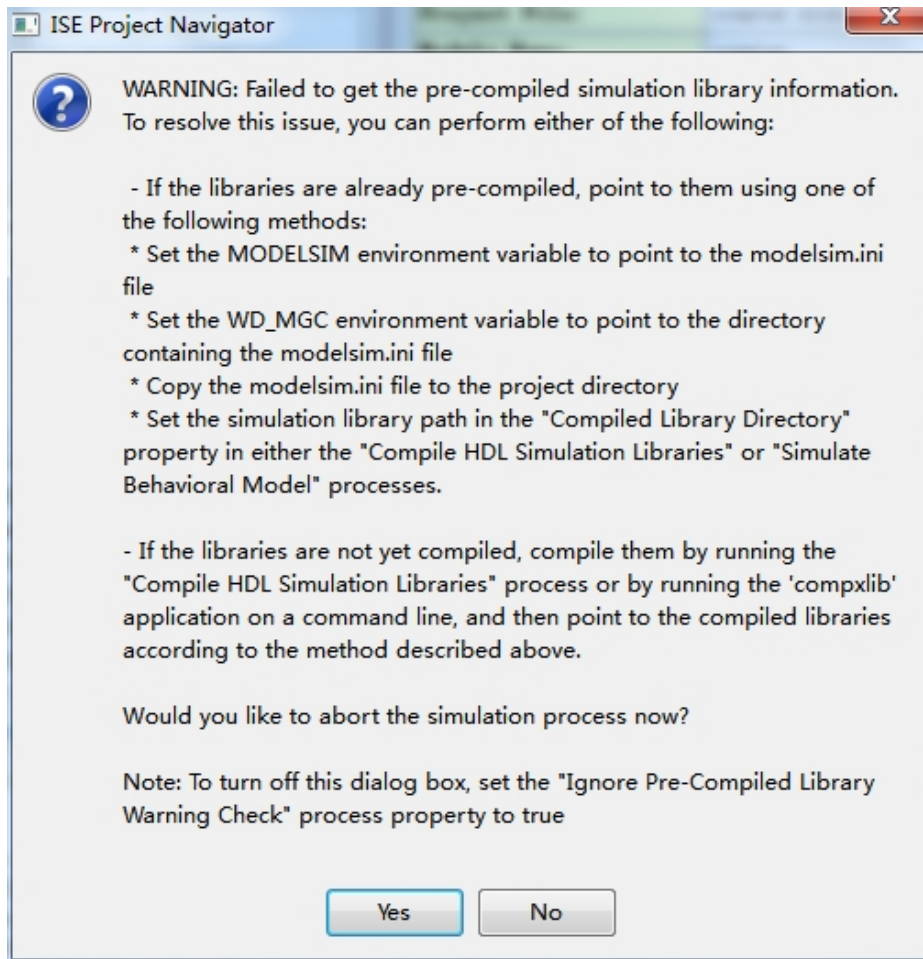
25 module counter_test;
26
27     // Inputs
28     reg reset;
29     reg clk;
30
31     // Outputs
32     wire [3:0] out;
33
34     // Instantiate the Unit Under Test (UUT)
35     counter uut (
36         .out(out),
37         .reset(reset),
38         .clk(clk)
39     );
40
41     always #50 clk = ~clk; //generate clk
42
43     initial begin
44         // Initialize Inputs
45         reset = 0;
46         clk = 0;
47
48         // Wait 100 ns for global reset to finish
49         #100 reset=1;
50         #100 reset=0;
51         #2000 $finish;
52     end
53 end
54
55 endmodule

```

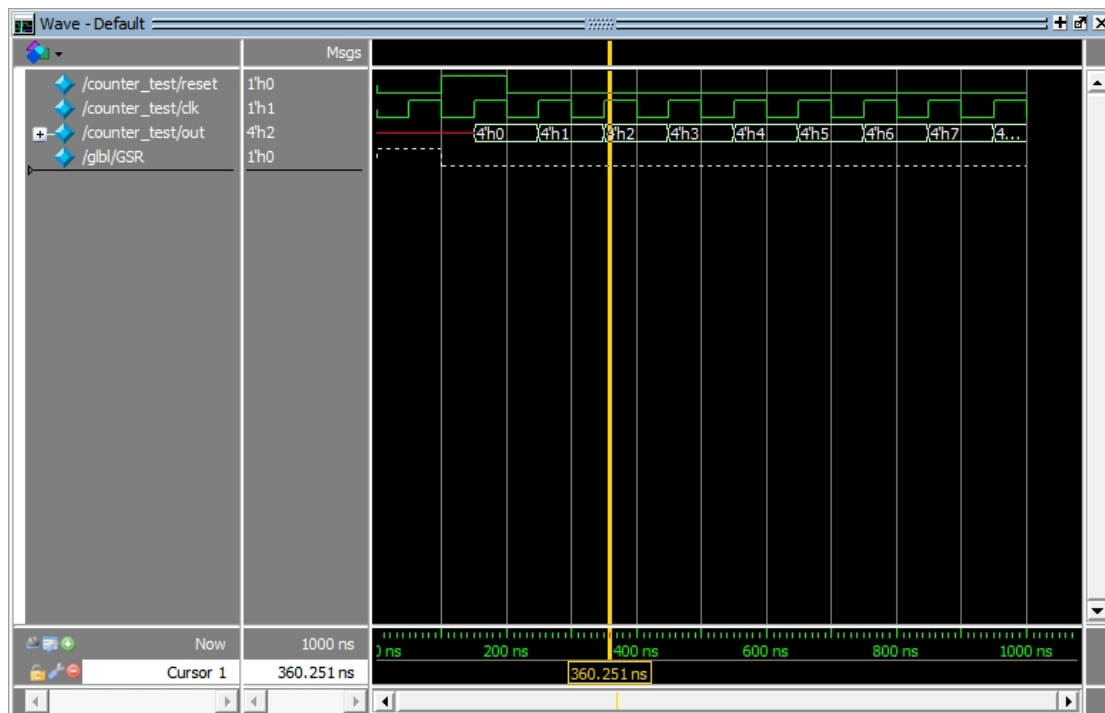
选中 testbench 文件，双击 Simulate Behavioral Model，调用 Modelsim 进行仿真。



出现下图，点击“NO”继续。



在 modelsim 的 wave 窗口，可查看到计数器波形，跟设计一致。



如果不一致，其他的操作参照第四章。